Can Large Language Models Identify and Refactor Code Clones? An Empirical Study

Xing Qian[a], Eman Abdullah AlOmar[a,*]

[a]*Stevens Institute of Technology, Hoboken, NJ, USA*

Abstract

Large Language Models (LLMs), such as GPT-4, have been widely used in various software engineering tasks, including code generation, code clone detection, refactoring and code review. Recent studies have compared the capability of different LLMs on code clone detection, especially the more complex Type-4 clones. However, despite all those studies, little is known about how LLMs perform on clone classification and refactoring compared to the state-of-the-art tools. In this paper, we explore how is GPT-4o's performance when we consider the results of traditional method JDeodorant as ground truth. We categorized clones into different scenarios, such as whole method clones within the same file, whole method clones in different files, code fragment clones within the same file and code fragment clones in different files. Then we compared the refactoring capability of LLMs, on different scenarios with JDeodorant based on compilation rate, correctness and the use of lambda expressions. We further expanded our evaluation to include a diverse set of LLMs, including Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7, and Claude Opus 4.1, along with traditional tools such as JDeodorant and AntiCopyPaster. These models and tools were applied to 20 case studies from the JHotDraw project, covering both Type-1 and Type-2 code clones. Our results show that GPT-4o achieved the perfect performance to identify Type-1 clones, but the accuracy dropped to around 51.2% for Type-2 clones and around 85.6% for Type-3 clones. Our findings reveal that while GPT-4o performs well on Type-1 clone classification, it struggles to identify Type-2 and Type-3 clones. For refactoring, GPT-4o achieved good results (84.2% correctness rate) when refactoring Type-1 clones. However, when clones are Type-2 or Type-3, the correctness of GPT dropped significantly, where it only has 54.6% for Type-2 and 33.1% for Type-3, indicating that GPT-4o struggles to refactor complex cases. When it comes to our case study, our results reveal a clear contrast between traditional tools and LLMs. JDeodorant and AntiCopyPaster consistently produced correct refactorings, with all clones detected, compiled, and tests passed. Among LLMs, GPT-4o and GPT-5 stood out, achieving 14 and 15 correct refactor-

*Corresponding author

Email addresses: xqian7@stevens.edu (Xing Qian), ealomar@stevens.edu (Eman Abdullah AlOmar)

ings respectively, while models such as Gemini 2.5 Pro, Claude Sonnet 3.7, and Claude Opus 4.1 showed partial success but often generated incomplete or fragmented changes. DeepSeek-R1 produced mixed results, sometimes replacing clones with calls but also introducing fragmentation of logic. In contrast, GPT-3.5 failed to detect all the clones. We anticipate our work could contribute to a boarder understanding of the capabilities of LLMs on code clone detection and refactoring.

Keywords: software quality, chatgpt, code clone, refactoring

---

1. Introduction

Artificial Intelligence (AI) has been increasingly applied in software engineering tasks, which help achieve great advancements in various domains. Among these achievements, Large Language Models (LLMs) have played a crucial role and demonstrated strong capabilities in code generation [1], automated testing [2, 3], bug fixing [4], and software refactoring [5, 6, 7]. In particular, code clone detection has been a popular area of exploration for Large Language Models (LLMs) [8, 9, 10], and refactoring has also recently received attention [11, 12, 5, 6, 7]. Code clones, which copy and reuse pieces of code, are common practices in software development. While it can boost productivity in some cases, too much cloning can make the code harder to maintain and increase the risk of spreading bugs [13]. Code refactoring is the practice of improving the internal structure of code without changing its external behavior [14]. The example of code clone and refactoring is provided in our replication package [15].

The rapid developments in AI and LLMs have offered great opportunities to improve this process. As LLMs continue to evolve, their capability to do code clone detection and refactoring has been an interesting area to explore. Recent studies have explored LLMs on code clone detection. For example, Zhang and Saber [10] compared GPT-3.5 and GPT-4 for clone detection and found that GPT-4 outperformed GPT-3.5, especially in detecting more complex clones. Similarly, Khajezade et al. [6] investigated the capability of ChatGPT and fine-tuned models, such as CodeBERT, RoBERTa in detecting mono-lingual and cross-lingual clones. Their results show that ChatGPT performed similarly with fine-tuned models in mono-lingual clone detection, but it surpassed them in cross-lingual clone detection. Additionally, Moumoula et al. [16] explored the comparison between LLMs and embedding-based approaches. Their results demonstrated that LLMs can perform well on simple clones, but the performance dropped significantly when dealing with complex cases. While these studies have provided insights into LLMs' effectiveness in code clone detection, they mainly focused on clone detection rather than clone type classification and refactoring. Moreover, the performances of LLMs on code clone refactoring across different scenarios, such as clones in the same file, clones across different files remain unexplored.

Traditional tools have been widely used for clone detection [17]. These tools utilized text-based [18], token-based [19] and abstract syntax tree [20]. As for refactoring, tools like JDeodorant [21, 22, 23] and WitchDoctor [24] are also well-known among developers. Among different refactoring types, extract method refactoring regularly appears in the top five most frequently used in real-world scenarios [25, 26, 27]. Current tools follow software quality metrics [28, 29, 30, 22, 31], but their suggestions often do not match real-world Extract Method practices. That is because effective refactoring needs both technical precision and human judgment. Developers draw not only on principles but also on their sense of what fits the code context—something most tools still struggle with. This gap likely contributes to developers' hesitation to adopt automated refactoring [32].

On the other hand, LLMs like GPT-4o [33] have demonstrated reasoning abilities to solve complex programming tasks [34, 35, 36]. But their ability to generate compilable and correct refactored code for clones has not been evaluated. This study aims to address these limitations by evaluating GPT-4o's abilities to detect and refactor clones. We also compared seven contemporary LLMs with traditional tools JDeodorant and AntiCopyPaster to evaluate whether LLMs can demonstrate consistent capabilities.

In our study, we studied if LLMs can correctly classify the clone type and apply refactoring. Since we used the output of JDeodorant as our ground truth and the refactoring as our baseline, our work primarily targeted Type-1 (Identical except for whitespace, comments, layout), Type-2 (Identical except for variable names, literals, types), and Type-3 (Mostly similar but with some different statements) clones, which are consistent with the types of clones that JDeodorant analyzes. We proposed an approach to analyze four distinct scenarios: (1) whole method clones within the same file, (2) whole method clones across different files, (3) code fragment clones within the same file, and (4) code fragment clones across different files. We applied prompt engineering techniques, including zero-shot, one-shot and few-shot learning to assess how they impact code clone classification accuracy. Furthermore, we manually evaluated the correctness and quality of refactored code based on compilation success, functionality preservation (via test cases) and use of lambda expression. We also designed case studies based on JHotDraw project. We tested JDeodorant, AntiCopyPaster and 7 LLMs, including Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1. We then evaluated their performances based on clone detection and correct "Extract Method" refactoring. Our dataset can be found in the replication package [15].

2. Related Work

In LLM-based code clone detection, Zhang and Saber [10] investigated the capability of GPT-3.5 and GPT-4. They used BigCloneBench (human-made) and GPTCloneBench (LLM-generated) datasets, employed few-shot prompting technique to enhance accuracy. Their results show that GPT-4 outperforms GPT-3.5 in code clone detection, especially in more complex clones. Khajezade

et al. [6] compared ChatGPT (GPT-3.5-turbo) with fine-tuned models, such as CodeBERT, RoBERTa, and GraphCodeBERT for mono-lingual (Java-Java) and cross-lingual (Java-Ruby) clone detection. They found that while ChatGPT achieved similar performances with fine-tuned models in mono-lingual settings, it outperforms them in cross-lingual settings. Due to the use of multiple programming languages, cross-lingual code clone detection has been essential. Moumoula et al. [16] studied four LLMs, including Falcon-7B, LLAMA2-Chat-7B, StarChat-Beta and GPT-3.5-turbo, with eight prompts, comparing with an embedding-based approach leveraging OpenAI's Text-Embedding-Ada-002 model for detecting cross-lingual code clones. Their findings indicate that GPT-3.5-turbo achieved the best result on simple cases, but its performance drops significantly when it comes to complex samples. The embedding-based approach outperforms LLMs on the XLCoST and CodeNet datasets. Nashaat et al. [37] analyzed transformer-based approaches to code clone detection. They proposed CloneXformer which leverages language-agnostic features and token-based representation. Their results show that CloneXformer outperformed state-of-the-art techniques by 16.88% F1-score on BigCloneBench. While LLMs have great potential in code clone detection, their performance is poor for Type-4 clones. To overcome this, researchers have proposed fine-tuning approaches. Inoue and Higo [38] executed fine-tuning on LLMs (GPT-3.5-turbo, Llama2-Chat-7B, CodeLlama-7B-Instruct) using FEMPDataset. Their results show that fine-tuning greatly improved the precision and overall accuracy. Notably, fine-tuned GPT-3.5-turbo outperformed GPT-4o and CodeLlama-7B-Instruct outperformed the base version. Xu et al. [39] introduced MANTRA, a comprehensive agent-based framework designed to automate method-level refactoring tasks by integrating Context-Aware Retrieval-Augmented Generation (RAG), coordinated multi-agent collaboration, and verbal reinforcement learning. MANTRA achieved 82.8% success rate in producing code that compiles and passes tests, compared to an 8.7% success rate by the baseline RawGPT model.

All of the above-mentioned studies have focused on evaluating the capability of LLMs on code clone detection or code generation and refactoring. Our work is different from these studies as our main purpose is to measure the performance of LLMs on not only the clone type classification, but also the refactoring. We leveraged the results of JDeodorant [21], an Eclipse plug-in which can help detect code clones and do refactoring by method extraction, as our datasets. Although JFreeChart was not originally designed as a benchmark dataset, in our study it serves as a benchmark, since its real-world clone instances and comprehensive test suite provide a consistent and reproducible basis for evaluating clone classification and refactoring across both LLMs and traditional tools. We compare GPT-4o with JDeodorant. We also evaluated different prompting techniques such as zero-shot, one-shot and few-shot learning. For code clones and refactoring, we test four scenarios, whole method clones in the same file, whole method clones in different files, code fragment clones in the same file and code fragment clones in different files. Our goal is to study if the format and different positions of clones would have impacts on the capability of LLMs in code clone refactoring. To the best of our knowledge, no previous study has investigated

the refactoring capability of LLMs (GPT-4o) on the format and positions of the clones. Further, to build the gap of comparing LLMs used, we also conducted a case study on the JHotDraw project. In this study, we compared several state-of-the-art LLMs, including Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, Claude Sonnet 3.7, Claude Opus 4.1, and DeepSeek-R1, alongside the traditional tool JDeodorant and AntiCopyPaster. In the next section, we detail the steps we took to design our setup.

3. Study Design

3.1. Research Questions

- **RQ1.** *How well can LLMs classify code clones?*

  Code clone detection and clone type classification can help developers identify redundant code. As LLMs offer a new approach, it is still unclear how well LLMs can distinguish between different clone types (Type-1, Type-2, Type-3). This research question aims to evaluate the performance of LLMs in classifying the types of code clones.

- **RQ2.** *How well can LLMs refactor code clones?*

  LLMs have the potential to perform refactoring by understanding code context and structure. This research question investigates the capability of LLMs to correctly refactor clones, as it remains uncertain how effective can LLMs do the refactoring.

- **RQ3.** *How consistent are LLMs' clone identification and refactoring capabilities across different models compared to traditional approaches?*

  LLMs may perform differently when used for clone detection and refactoring, and it is not clear how stable their results are across models. This research question examines whether LLMs consistently yield reliable outcomes or if their performance varies significantly depending on the model. By comparing them with traditional tools, we can see where LLMs are reliable and where they may still fall short.

Table 1: JFreechart number of each clone overview

Scenarios	Type-1	Type-2	Type-3
Whole method clone in same file	0	463	29
Whole method clone in different files	78	2216	515
Code fragment clone in same file	72	784	60
Code fragment clone in different files	287	761	92
Duplication	37	508	75
Not in source folder	17	128	9

3.2. Open Source Project Selection

To select open-source projects for our study, we considered nine projects previously studied by Tsantalis et al. [21]. These projects are Apache Ant 1.7.0, Columba 1.4, EMF 2.4.1, JMeter 2.3.2, JEdit 4.2, JFreeChart 1.0.10, JRuby 1.4.0, Hibernate 3.3.2 and SQuirreL SQL 3.0.3. To fully understand each project and find the best projects suitable for our experiments, we conducted a detailed analysis of each project. We extracted clones from the reports of JDeodorant [21] to calculate the number of clones in different projects. Some of the reports contain multiple clones, which we initially mined as separate instances. However, we removed any clones located within the Test folder because the false negatives (clones which are not detected) of clone detection tools occur more frequently in test code than in production code [40]. Additionally, we excluded clones that were not located in the source folder. As outlined in our research questions, we aim to assess the capability of LLMs in performing code clone refactoring, test files are required to do the evaluation, so we excluded EMF 2.4.1 and JEdit 4.2 which do not have test files. In projects that include test files, we found that not all cloned methods were covered by tests. To address this, we used GPT-o3-mini-high[1], which had the highest ELO score for code completion, to generate the missing test methods. Among the remaining projects, we wanted to select the one that would provide the most comprehensive dataset for our experiments. Due to JFreeChart 1.0.10's significantly larger number of code clones, we selected it as the primary dataset for experiments. Table 1 summarizes the number of clones identified across different scenarios. We manually reviewed each clone. Two authors conducted the data analysis. One manually inspected the code changes, while the other reviewed the results to reach an agreement. During the manual review, we also came across duplicate clone pairs where two pairs had identical source folders, packages, classes, methods, and start/end lines. We simply skipped these duplicates during later inspections. Since we utilized API to run LLMs, the cost for running multiple projects is much higher than single project. Thus, we selected JFreeChart for our experiment in which its cost was around $1,500.

For our case study, we selected JHotDraw as our primary project. The reason is that JHotDraw is a mature and widely studied open-source Java framework that has been used in numerous software engineering studies on code clone [41, 42, 43, 44, 45]. We randomly selected 30 methods from the JHotDraw codebase and manually introduced clone instances. The number of selected methods was intentionally limited due to the manual effort required to validate the clone detection and refactoring results across all models. This process involved not only inserting realistic clones, but also thoroughly checking compilation outcomes and test case behaviors for each tool, which made scaling to a larger dataset impractical within the scope of this study. The introduced clone instances included Type-1 and Type-2. However, since Deckard was only capable of detecting Type-2(a) and Type-2(b) clones for Type-2, we limited the compar-

[1] https://openai.com/index/openai-o3-mini/

ison among the three tools to Type-1, Type-2(a), and Type-2(b) clones. After filtering out cases, the total number of methods used in the final comparison was 20. We found that the test files did not fully cover the cloned methods, we utilized GPT-o4-mini-high[2] to generatetest methods because it achieves remarkable performance in coding tasks [46].

3.3. Large Language Models Selection

To select the LLMs most suitable for our study, we first examined models from previous study [3]. Among the candidates, we considered StarCoder [47] and Codex[3] because they were primarily trained on code repositories and optimized for code completion and generation. However, they lack the advanced reasoning capabilities required for clone type classification and clone refactoring. When providing natural language along with cloned code as input prompts to StarCoder, the model often generates unrelated code instead of indicating the type of provided code fragments as responses. We also considered Gemini models [48] since their API has a free tier. However, their free tier has limited tokens and requests. Given those limitations, we found that general-purpose LLMs such as GPT models performed better in previous studies [6, 16] , as a result, we selected GPT-4o for our study.

To run GPT-4o on our end, we utilized Azure OpenAI[4] and accessed the model with API calls. To evaluate whether different prompting techniques would affect the performance of LLMs, we explored zero-shot, one-shot, and few-shot learning. The entire experimental process took 12 days , including clone type classification and refactoring. The total cost of API usage calling was $1,500.

Further, in our case study, we selected seven contemporary models, including Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1. For all LLM-based tools, we selected their most recent publicly available versions to ensure a fair and up-to-date comparison, except GPT-3.5 and Claude Sonnet 3.7.

3.4. Traditional Tool Selection

Despite JDeodorant, in our case study for RQ3, we selected AntiCopyPaster [49, 50, 51, 52] as another traditional tool as a baseline. AntiCopyPaster is an IntelliJ-based plugin designed to automatically detect and refactor code clones in real time. It provides just-in-time "Extracted Method" refactoring suggestions immediately after duplicate code is pasted, helping developers eliminate redundancy without interrupting their development workflow. The tool uses both semantic and syntactic code metrics to decide whether a code fragment should be extracted and supports different types of clones, including Type-1 and Type-2.

[2] https://openai.com/index/introducing-o3-and-o4-mini/
[3] https://openai.com/index/openai-codex/
[4] https://azure.microsoft.com/services/openai-service/

We selected AntiCopyPaster as our traditional baseline because it represents a mature, IDE-integrated refactoring tool that combines clone detection and automated method extraction. Unlike LLMs, which rely on text-based reasoning and generation, AntiCopyPaster's rule-based and PSI-based (Program Structure Interface) approach provides a stable and deterministic comparison point. This allows us to assess how consistently LLMs can achieve human-level clone detection and refactoring quality under a well-established traditional setting.

3.5. Prompt Engineering

Prompt engineering plays a crucial role in optimizing LLMs. As mentioned in Section 3.3, we applied zero-shot [53, 54], one-shot [55] and few-shot learning [56] techniques for clone type classification to evaluate whether different prompting strategies would improve the performance.

In the zero-shot setting, we started with a system instruction, which set LLM as a Java developer. Following this we provide definitions of each clone type. Although our primary goal is to study Type-1, Type-2 and Type-3 clones, we still include the definition of Type-4 clones to evaluate whether the LLM can classify types correctly. Next, we include the method information in the prompt, such as method name and its start and end line number. Those details are provided in the previous study [21]. Additionally, we read the whole corresponding Java file and insert the content in the prompt. Listing 1 shows what our prompt for zero-shot looks like.

```
You are a Java developer with expertise in code clone detection and refactoring.

Here are the definitions of clone types:
Type-1: Identical except for whitespace, comments, layout
Type-2: Identical except for variable names, literals, types
Type-3: Mostly similar but with some different statements
Type-4: Different structure but same functionality
{content}
I have uploaded a Java source code content.
Please extract and analyze the following two code snippets from the content:
Code snippet 1- Method: {['Method␣1']} - Lines: {['Start␣Line␣1']} - {item['End␣Line␣1']}
Code snippet 2- Method: {item['Method␣2']}
- Lines: {['Start␣Line␣2']}-{['End␣Line␣2']}
With no explanation, directly print these two code snippets, also indicate what types of
     clones are in these two code snippets.
```

Listing 1: Prompt for zero-shot

In the one-shot and few-shot setting, we provided examples for each clone type. We selected our examples from representative samples that define each clone type. The complete list of examples is provided in our replication package [15].

We randomly selected 30 clones for manual analysis. Using prompts we designed above, we queried GPT-4o model integrated in ChatGPT to perform clone type classification, then evaluated the performances by comparing the clone type generated by GPT with JDeodorant. Our analysis shows that one-shot and few-shot prompt engineering techniques did not improve the results compared to zero-shot learning. Given the increased computational cost and

time required for one-shot and few-shot approach, we decided to use zero-shot in our experiments.

3.6. Clone Type Classification

Since Tsantalis et al. [21] primarily focused on Type-1, Type-2, and Type-3 clones, our study also follows this approach, concentrating on these three clone types to ensure a consistent alignment with prior research. Tsantalis et al. [21] utilized four clone detection tools, including CCFinder [57], Deckard [58], CloneDR [20] and NiCad [59]. They found that Deckard tends to detect more Type-2 and Type-3 refactorable clones, and also refactorable clones it found tend to have larger size. Among those clone detection tools, Deckard is the one that mostly reports complete Abstract Syntax Tree (AST) statements as clones. So in our study, we used Deckard clone results.

Furthermore, we divided our dataset into four scenarios. Specifically, we considered two cases regarding code clone types:

- Whole method clones - where the entire method is identied as a clone.
- Code fragment clones - where only a portion of a method is a clone.

We also considered two cases regarding the position of code clones:

- Same class clones - clones are within the same file.
- Different files clones - clones are in different files.

To classify those cases, we utilized HTML reports of JDeodorant results, which provide clone detection results along with the corresponding source folder, package, class and line numbers. We applied regular expression to mine the cloned code fragments and their containing methods in JFreeChart project. Finally, we compared the extracted code fragments with the methods to determine whether the clones are whole method clones or code fragment clones. Therefore, we have 4 cases in total, whole method clones in different files, whole method clones within the same file, code fragment clones in different files and code fragment clones within the same file.

Since there were four scenarios, we designed different prompts for different scenarios of code clone.

```
You are a Java developer with expertise in code clone detection and refactoring.

Here are the definitions of clone types:
Type-1: Identical except for whitespace, comments, layout
Type-2: Identical except for variable names, literals, types
Type-3: Mostly similar but with some different statements
Type-4: Different structure but same functionality
Now I have two code snippets, one is in method {['Method␣1']} with start line
     {item['Start␣Line␣1']} and end line {item['End␣Line␣1']} of following java class
{content_1}
Another one is in method {['Method␣2']} with start line {['Start␣Line␣2']} and end line
     {['End␣Line␣2']} of following java class:
{content_2}
With no explanation, directly print these two code snippets, also indicate what types of
     clones are in these two code snippets.
```

Listing 2: Prompt for whole method clones detection in different files

Prompts for whole method clones are shown in Listing 1, Listing 2. Each prompt starts with a system instruction, which set LLM as a Java developer. Following this we provide definitions of each clone type. Although our primary goal is to study Type-1, Type-2 and Type-3 clones, we still include the definiton of Type-4 clones to evaluate whether the LLM can classify types correctly. Next, we include the method information in the prompt, such as method name and its start and end line number. Those details are provided by JDeodorant's report. Additionally, we read the whole corresponding Java file and insert the content in the prompt. This approach ensures the LLM have the complete context to analyze the clones, which is crucial for the refactoring part in the later step. Finally, we instruct LLM to print out the code snippet it finds to confirm it finds the correct code snippets before classifying.

```
You are a Java developer with expertise in code clone detection and refactoring.

Here are the definitions of clone types:
Type-1: Identical except for whitespace, comments, layout
Type-2: Identical except for variable names, literals, types
Type-3: Mostly similar but with some different statements
Type-4: Different structure but same functionality
Given the two code snippets below, without any additional explanation, determine and
     return the appropriate clone type classification:
Code snippet 1:
    ['Code␣fragment␣1']
Code snippet 2:
    ['Code␣fragment␣2']
```

Listing 3: Prompt for code fragment clones detection

Prompt we used for code fragment clones type classification is provided in the Listing 3. We kept the system prompt and definitions of each clone type. However, instead of telling the model method name, start line, end line and java file content, we directly inserted code fragments to the prompts because code fragments are shorter than the whole methods. Then we asked the model to directly output the clone type like what we did for whole methods clones.

3.7. Code Clones Correction

Following the clone type classification step, we study the refactoring further. We store both the prompt and response from the classification phase, to ensure continuity.

The refactoring prompts for whole method clones are provided in Listing 4.

```
Refactor the clones in the given two code snippets by extracting the clones into a
     single method.
Ensure that there are no compilation errors and the functionality remains unchanged.
Provide both:
1. The extracted method.
2. The method calls that should replace the clones in the original files.
Do not include any explanations, only provide the refactored code.
```

Listing 4: Prompt for whole method clones refactoring within the same file

```
Refactor the clones in the given two code snippets by extracting the clones into a
     single method.
Ensure that there are no compilation errors and the functionality remains unchanged.
```

```
- Since the clones exist in different files, extracted method will be written into their
     common superclass.
- Ensure that instance-specific variables (e.g., `this.x`) are passed as parameters to
     the extracted method to maintain functionality
Provide both:
1. The extracted method.
2. The method calls that should replace the clones in the original files.
Do not include any explanations, only provide the refactored code.
```

Listing 5: Prompt for whole method clones refactoring in different files

Since we use JDeodorant's results as our ground truth, as shown in JDeodorant [60], approximately 49.6% of the total refactorings recommended are done by "Extract Method", we use the same approach "Extract Method" to refactoring. "Extract Method" is one of the most important refactorings [61]. This refactoring was among the first to be formally defined in the field, as it offers a practical way to keep methods short, modular, and free from overlapping responsibilities [62]. In addition, "Extract Method" often acts as a stepping stone for carrying out more complex refactorings [63]. Both open-source developers [64, 65, 66, 67, 28, 68, 25, 69] and industry practitioners [70] regard it as a key operation. Its popularity comes from its versatility: developers apply it for removing duplicated code [71, 72, 73, 74], extracting reusable functionality [64, 75, 76], adapting outdated method signatures [64], simplifying large or complicated logic [31, 77, 30, 78, 79, 80, 81], and improving code testability [82, 83]. Because it supports such a wide range of scenarios, Extract Method is often described as the "Swiss Army knife" of refactoring [84]. In our study, we instruct the GPT-4o to refactor the clones by extracting them into a single method, also ensuring that the refactored code compiles correctly and maintains the functionality. The only difference between prompts for same file (Listing 4) and different files (Listing 5) is how the extracted method is handled. For clones in different files, the method is extracted to the common superclass. To ensure correctness, we instruct the GPT-4o to pass class instance variables as parameters in the extracted method. We believe including this instruction in the prompt is necessary, as the GPT-4o does not have access to the full project structure. We want to ensure a fair and consistent approach by guiding LLMs in this way.

The prompts used for refactoring code fragment clones (see Listing 6, Listing 7 and closely resemble those for whole method clones, with a slight adjustment. Specifically, when classifying code fragment clone types, the original Java file content was not provided. Therefore, we add the file content into the refactoring prompts for clarity. Because of the large amount of clones and the time consuming of manual verification, we selected a samples size to achieve a 95% confidence level and 5% margin of error, as shown in Table 2.

```
Both given code snippets are in following java file:
{content1}
Refactor the clones in the given two code snippets by extracting the clones into a
     single method.
Ensure that there are no compilation errors and the functionality remains unchanged.
Provide both:
1. The extracted method.
2. The method calls that should replace the clones in the original files.
```

```
Do not include any explanations, only provide the refactored code.
```

Listing 6: Prompt for code fragment clones refactoring within the same file

```
The first code snippet is in following java file:
{content_1}
The second code snippet is in following java file:
{content_2}
Refactor the clones in the given two code snippets by extracting the clones into a
     single method.
Ensure that there are no compilation errors and the functionality remains unchanged.
- Since the clones exist in different files, extracted method will be written into their
     common superclass.
- Ensure that instance-specific variables (e.g., `this.x`) are passed as parameters to
     the extracted method to maintain functionality
Provide both:
1. The extracted method.
2. The method calls that should replace the clones in the original files.
Do not include any explanations, only provide the refactored code.
```

Listing 7: Prompt for code fragment clones refactoring in different files

Table 2: Number of evaluated cases in each scenario

Scenarios	**Numbers**	**Type-1**	**Type-2**	**Type-3**
Whole method clones within the same file	Total number	0	463	29
	Evaluated number	0	211	29
Whole method clones in different files	Total number	78	2216	515
	Evaluated number	70	328	221
Clone fragment clones within the same file	Total number	72	784	60
	Evaluated number	62	262	55
Clone fragment clones in different files	Total number	287	761	92
	Evaluated number	151	249	88

Note: The total number refers to all reported clones, including duplicates. The evaluated number represents the subset of clones we manually assessed, determined using a 95% confidence level and a 5% margin of error. This evaluated count is equal to or greater than the minimum sample size required based on the total number of clones.

3.8. Qualitative Analysis

To evaluate the correctness of the refactoring, we manually integrated the generated refactoring code into the JFreeChart project using Eclipse IDE. Specifically, we replaced the original cloned methods or code fragments with appropriate method calls pointing to the newly extracted methods. If clones were located in different classes, we first checked to see if these classes shared a common superclass. If such a superclass existed, we placed the extracted method there. However, if no common superclass was found, we manually created a separate utility class. This new class stored the extracted methods, and we then imported this utility class into the original classes, allowing both to reference the extracted method efficiently. After integrating each refactored method, we carefully checked for compilation errors. Any refactoring causing compilation

issues was immediately marked as incorrect and excluded from the next step in our evaluation.

To verify that the refactored methods maintained the original functionality, we next looked for existing JUnit test files associated with the modified classes. If no relevant test file was available, we discarded those refactoring cases, as validating their functionality would not be feasible without proper tests. For classes where a test file existed, we closely examined the tests to determine whether they specifically covered the functionality of the extracted cloned methods. In cases where existing tests did not adequately cover these methods, we leveraged GPT-o3-mini-high[5], which achieved the highest ELO score for code completion among OpenAI models, to generate appropriate unit tests. We manually reviewed and validated these generated tests to ensure their accuracy and comprehensiveness before incorporating them into the test suite.

Finally, we executed the complete JUnit test suite, carefully observing the results to confirm that all tests passed successfully. Refactoring cases that resulted in failing tests were considered incorrect. Through this rigorous manual integration, compilation checking, targeted test generation, and comprehensive testing process, we ensured a robust and reliable evaluation of the refactoring correctness.

3.9. Statistical Analysis

We applied statistical analysis to study the models and tools performance to see if they vary. To rigorously assess whether the observed performance differences between models and tools are statistically significant, we employ McNemar's test [20, 85], a non-parametric statistical test designed for paired nominal data. McNemar's test is particularly well-suited for our analysis because it evaluates whether two classification methods (in our case, different models or tools) perform differently on the same set of test cases. The test focuses on cases where the two methods disagree, examining whether one method systematically outperforms the other or if the disagreements are balanced. Unlike unpaired tests that assume independence between samples, McNemar's test accounts for the paired nature of our data, where each clone instance is evaluated by both methods. The test statistic follows a chi-squared distribution with one degree of freedom, and a significant result ($p < 0.05$) indicates that the two methods exhibit statistically different performance patterns.

For each research question, we applied McNemar's test. For RQ1, we compared the performance of GPT-4o and JDeodorant on code clone classification. For RQ2, we excluded JDeodorant cases with precondition violations and retained only those that could be refactored. We then compared compilation success, test pass rates, and correctness of refactoring. In evaluating test pass rates, we considered only correctly compiled cases and excluded those that failed to compile. For RQ3, we conducted pairwise comparisons among the six models that produced clone detections, excluding GPT-3.5 since it did not detect any

[5] https://openai.com/index/openai-o3-mini/

clones. Here, we focused solely on correctness of refactorings, as all models were able to detect clones and produce refactorings that compiled and passed test cases.

4. Experimental Results

Table 3: Overview of classification results for GPT-4o

Scenarios	Clone Type	Results					
		Type-1	Type-2	Type-3	Type-4	No clone	Wrong method extracted
Whole method clones within the same file	Type-1	0	0	0	0	0	0
	Type-2	13 (6.9%)	94 (49.7%)	82 (43.4%)	0	0	22
	Type-3	0	5 (20.8%)	19 (79.2%)	0	0	5
Whole method clones in different files	Type-1	68 (100.0%)	0	0	0	0	2
	Type-2	12 (3.8%)	179 (56.6%)	124 (39.2%)	1 (0.3%)	0	12
	Type-3	1 (0.5%)	17 (7.7%)	202 (91.4%)	1 (0.5%)	0	0
Clone fragment clones within the same file	Type-1	62 (100%)	0	0	0	0	0
	Type-2	3 (1.1%)	134 (51.1%)	125 (47.7%)	0	0	0
	Type-3	2 (3.6%)	5 (9.1%)	47 (85.5%)	1 (1.8%)	0	0
Clone fragment clones in different files	Type-1	151 (100.0%)	0	0	0	0	0
	Type-2	12 (4.9%)	112 (45.3%)	123 (49.8%)	0	0	2
	Type-3	3 (3.8%)	13 (16.3%)	56 (70.0%)	7 (8.8%)	1 (1.3%)	8

Note: The percentages in parentheses represent the proportion of predictions within each clone scenario and clone type. For example, in the second row, 13 predictions (6.9%) were labeled as Type-1 clones out of a total of 189 predictions for Type-2 clones within the same file. These percentages are calculated row-wise to show how GPT-4o distributed its predictions across different clone categories for each scenario. A higher percentage in a particular column indicates GPT-4o's stronger tendency to classify clones into that category under the given condition.

4.1. Clone Type Classification

Results. Table 3 summarizes the overall accuracy of clone type classification across different scenarios and clone types. The results clearly indicate that Type-1 clones were consistently classified with perfect accuracy in every scenario tested. It shows the model's strong capability in recognizing identical or nearly identical code. However, the accuracy drops when it comes to Type-2 and Type-3 clones, which indicates more challenges as code variations become more significant. Specifically, accuracy for Type-2 clones ranged around approximately 50%, with the lowest accuracy at 45.3% for code fragment clones in different files and the highest accuracy at 56.6% when identifying whole method clones across different files. For Type-3 clones, the accuracy varied depending on the scenarios. It achieved its highest score of 91.4% for whole method clones in different files, while dropping to around 72% to 85% in other cases, which suggests the model struggles as code variations increase. We excluded Type-4 clones from our analysis because JDeodorant, which we used as our ground truth, primarily targets Type-1, Type-2, and Type-3 clones.

Whole method clones in same file. As shown in Table 3, With Type-2 clones, the model was able to identify 94 total. Unfortunately, 82 were misidentified as Type-3 clones, indicating an issue when distinguishing moderate changes. There were also 13 Type-2 clones misclassified as Type-1. On the other hand, GPT-4o correctly found 19 clones for the Type-3 category which has higher degree of structural difference between them. However, the model also misassigned 5 correct Type-3 clones as Type-2, reinforcing the problem it encountered with differentiating the two categories. In the end, the model did not do well in separating the clones with moderate differences.

Whole method clones in different file. For whole method clones across different file types, the results show high accuracy in correctly identifying Type-1 clones, with 68 correctly classified instances. However, performance declines with more complex clone types. There are 179 correctly identified cases for Type-2 clones, but also many instances misclassified, where 124 were mislabeled as Type-3 and 12 were misclassified as Type-1. There were even one Type-2 clone identified as Type-4 clone incorrectly. Classification accuracy is higher for Type-3 clones than Type-2 clones, with 202 correctly identified, alongside 17 misclassified as Type-2, 1 as Type-1 and 1 as Type-4.

Code fragment clones in same file. The classification results for different types of code fragment clones within the same file show that Type-1 clones were identified very effectively, with all 62 instances correctly classified and no misclassifications. GPT-4o also successfully identified 134 Type-2 clones. However, it misclassified 3 of these as Type-1 and 125 cases as Type-3, which also suggests some confusion between Type-2 and Type-3 categories. For Type-3 clones, the model classified 47 cases accurately, though it mistakenly labeled 2 as Type-1 and 5 as Type-2. Interestingly, there was also one case where a Type-3 clone was incorrectly classified as Type-4. Overall, while the model is good at classifying clear-cut Type-1 clones, it shows some challenges differentiating between clones of Type-2 and Type-3.

Table 4: Overview of refactoring results for GPT-4o

Scenarios	Clone Type	Results									
		Correct	Compilation error	Test failes	Wrong codes extracted	No test files	No method calls	Incomplete refactoring	Expanded refactoring	Comprehensive refactoring	JDeodorant
Whole method clones within the same file	Type-1	0	0	0	0	0	0	0	0	0	0
	Type-2	163 (89.6%)	17 (9.3%)	2 (1.1%)	22	7	0	0	0	0	136
	Type-3	8 (53.3%)	6 (40.0%)	1 (6.7%)	5	9	0	0	0	0	4
Whole method clones in different files	Type-1	47 (71.2%)	16 (24.2%)	3 (4.5%)	2	2	0	0	0	0	62
	Type-2	113 (36.5%)	142 (45.8%)	55 (17.7%)	12	0	6	0	0	0	127
	Type-3	50 (25.0%)	67 (33.5%)	82 (41.0%)	0	1	20	1 (0.5%)	0	0	5
Clone fragment clones within the same file	Type-1	52 (83.9%)	9 (14.5%)	0	0	0	0	1 (1.6%)	1	1	57
	Type-2	180 (62.9%)	49 (17.1%)	6 (2.1%)	0	0	2	51 (17.8%)	5	8	188
	Type-3	31 (56.4%)	17 (30.9%)	3 (5.5%)	0	0	0	4 (7.3%)	0	5	7
Clone fragment clones in different files	Type-1	136 (90.1%)	10 (6.7%)	1 (0.7%)	0	0	0	4 (2.6%)	28	9	130
	Type-2	101 (41.6%)	111 (45.7%)	11 (4.5%)	2	0	1	20 (8.2%)	15	3	85
	Type-3	26 (32.5%)	40 (50.0%)	4 (5.0%)	8	0	0	10 (12.5%)	4	0	6

Note: Incomplete refactoring refers to cases where the extracted method either does not fully cover all the cloned code or still contains cloned segments. **Expanded refactoring** means the refactoring included additional lines beyond the original clone. **Comprehensive refactoring** describes situations where the entire method, not just the cloned fragments, was refactored. When calculating the percentage, we consider both **expanded refactoring** and **comprehensive refactoring** as correct if they do not introduce compilation errors and all test cases pass, since the targeted clones are successfully removed.

Code fragment clones in different files. Table 3 provides the classification results for code fragment clones located across different files. The results show impressive accuracy for Type-1 clones, as the model successfully identified all 151 instances. However, the classification performance dropped for Type-2 clones: the model correctly recognized 112 cases, but it misclassified 12 as Type-1 and 123 as Type-3. Type-3 clone classification faced similar challenges. While the model accurately identified 56 cases, it misclassified 3 as Type-1, 13 as Type-2, 7 as Type-4, and even one case was mistakenly marked as "No clone." These results highlight that although the model is highly effective with identical or nearly identical code fragments, it struggles more significantly when handling clones with high differences across files.

Observation: ***Misclassification cases.*** While GPT-4o demonstrated robust capabilities in Type-1 clone classification, several misclassifications were observed for Type-2 clones and Type-3 clones across different scenarios, especially for Type-2 clones. These inaccuracies highlight weaknesses in GPT-4o's ability to differentiate between the subtler variations present in Type-2 clones. The model frequently struggled to correctly classify Type-2 clones, especially when distinguishing them from Type-3 clones, resulting in significant drops in accuracy.

GPT-4o had particular trouble with Type-2 clones in two specific situations: whole method clones within the same file and fragment clones located in different files. In these scenarios, the accuracy fell below 50%, indicating the model found it challenging to correctly identify clones with moderate variations. We found that GPT-4o tends to depend heavily on structural differences that are clearer in Type-3 clones. As a result, when faced with Type-2 clones that are somewhat ambiguous or borderline, it often misclassified them as Type-3. This explains why Type-3 clones had consistently higher accuracy across different scenarios—the model defaults to labeling complicated or heavily modified code fragments as Type-3 rather than Type-2. Examples of Type-2 and Type-3 clones misclassification are provided in the replication package [15]. Both examples demonstrate the challenge that GPT-4o distinguishes Type-2 and Type-3 clones.

We also discovered 10 instances where GPT-4o incorrectly classified clones as Type-4. Among these 10 instances, 9 of them were actually Type-3 clones, meaning they still had similar structures. This suggests that GPT-4o might struggle to correctly identify complex clones when their differences are enough to blur the structural similarity. Specifically, when code fragments have high difference rates, the model tends to categorize them as Type-4 clones that have the same functionality but entirely different structures.

RQ1 Summary

Overall, GPT-4o performs well on classifying Type-1 clones. It correctly classified all the Type-1 instances. However, for Type-2 and Type-3 clones, GPT-4o's performance dropped significantly. The results show that GPT-4o struggles with distinguishing between Type-2 and Type-3 clones.

4.2. Code Clones Correction

Results. Table 4 provides an overview of refactoring results achieved by GPT-4o across different clone types and scenarios, along with the number of clones that JDeodorant can apply refactorings. The results show that Type-1 clones consistently had the highest correctness rates, reaching around 71% to 90%, indicating that GPT-4o handles identical or nearly identical code effectively when refactoring. However, performance declines significantly with Type-2 and Type-3 clones, which involve more code variations. For example, the correctness rate for Type-2 clones ranged broadly, from as low as 36.5% for whole method clones in different files to a higher rate of 89.6% for clones within the same file. Type-3 clones presented the greatest challenges, particularly in clone fragment refactoring across different files, where correctness dropped sharply to about 32.5%. Table 4 also shows the results of JDeodorant. Compared to GPT-4o, JDeodorant performed better when refactoring Type-1 and Type-2 whole method clones in different files and code fragment clones in the same file. In some cases, we found that GPT-4o refactors file despite precondition violations indicated by JDeodorant.

Whole method clones within the same file. The refactoring results in Table 4 for whole method clones identified within the same file shows that Type-2 clones had 163 successful refactorings, but with noticeable errors: 17 compilation errors, 2 test failures. Type-3 clones, though fewer in number, also display mixed results: 8 correctly refactored instances, with 6 compilation errors, 1 test failure.

Whole method clones in different files. The results of refactoring whole method clones located in different files vary significantly depending on clone complexity. Type-1 clones had the highest success rate, with 47 correctly refactored instances, though still encountering 16 compilation errors and 3 test failures. Refactoring for Type-2 clones faced challenges: while 113 instances were correctly refactored, they suffered from a high number of compilation errors (142) and test failures (55). Type-3 clones also had limited success with only 50 correct refactorings, with notable issues—67 compilation errors, 82 test failures and 1 incomplete refactoring.

Code fragment clones within the same file. Table 4 presents the results of refactoring code fragment clones within the same file. For Type-1 clones, the results were generally positive, with 52 correctly refactored cases and only 8 resulting in compilation errors. Notably, none failed testing or contained clones in extraction. Type-2 clones, though GPT-4o successfully refactored in many

cases (181 correct instances), it faced more issues, with a higher incidence of compilation errors (49 cases) and test failures (6 cases). We also observed 24 instances in which GPT failed to find common logic shared among the clones. Instead, it just simply included the entire clones in the extraction method, and then used conditional statements if-else to differentiate between them in the original clones. Type-3 clones proved to be the most challenging: only 33 were refactored successfully, while 17 led to compilation errors, 3 failed tests, and 2 still retained clones in the extraction.

Code fragment clones in different files. The refactoring results for code fragment clones found across different files are also provided in Table 4. Type-1 clones showed strong performance with 137 successfully refactored instances and relatively few errors, consisting of just 10 compilation errors and a single test failure. Type-2 clones presented more challenges, achieving 118 correct refactorings but accompanied by a significant number of compilation errors (111), test failures (11), and notably, one instance that still contained clones post-refactoring. Type-3 clones proved most challenging, achieving only 26 correct refactorings alongside 40 compilation errors and 4 test failures.

We evaluated the results of refactoring based on compilation rates, correctness, and the use of lambda expressions.

Compilation rates. Compilation errors constitute a critical factor in evaluating the effectiveness of code refactoring. To assess the compilation success of the refactored code, we checked whether the generated code compiled without errors. As we can see from Table 4, the compilation success rate varies significantly across clone types and scenarios. For Type-1 clones, compilation errors remain consistently low across scenarios. Specifically, clone fragment clones within the same file result in only 9 compilation errors (14.5%), and clone fragments in different files result in just 10 errors (6.7%). Notably, GPT-4o performs particularly well when handling whole method Type-2 clones within the same file, encountering compilation errors in only 17 cases (9.3%). However, its performance declines when refactoring whole method Type-2 clones across different files, with 142 cases having compilation errors (45.8%). Compared to Type-1 and Type-2, Type-3 clones are more complex, resulting in 6 compilation errors within the same file (40.0%) for whole methods and 17 for clone fragments (30.9%). Cross-file scenarios show increased complexity, leading to 67 compilation errors for whole method clones (33.5%) and 40 errors (50.0%)for clone fragment clones.

The table also highlights a significant increase in compilation errors when GPT-4o refactors clones in different files compared to scenarios where clones are in the same file. For clones in different files, the compilation rates drop for all clone types. When refactoring the clones in different files, since the extracted methods need to be placed in either the superclass or a separate file, GPT-4o generates code that includes variables or method references which do not exist in those locations. This leads to compilation errors because the required context or definitions are missing. For instance, clone group 93-1-2 contains Type-3 clones in different files. When GPT-4o extracted method, it included that.itemLabelGenerator variable (see Listing 8). Since this variable was not

defined in the superclass AbstractRenderer, the extracted method caused a compilation error.

```
AbstractRenderer that = (AbstractRenderer) obj;
 if (!ObjectUtilities.equal(itemLabelGenerator, that.itemLabelGenerator )) {
          return false;
      }
```

Listing 8: Undefined variable compilation error: refactoring of clone group 93-1-2 (highlighted with yellow)

There is also another common compilation error, where the types of parameters in the extracted method do not match with their actual types. For example, when refactoring clone group 1172-1-3, as we can see from Listing 9, the variable domainAxisLocations was defined as ObjectList type.

```
private ObjectList domainAxisLocations;

//Method signature of extract method
protected void updateDomainAxisLocation(int index, AxisLocation location, boolean notify,
     List<AxisLocation> domainAxisLocations)

//Method call
updateDomainAxisLocation(index, location, notify, this.domainAxisLocations);
```

Listing 9: Unmatched types compilation error: refactoring of clone group 1172-1-3 (highlighted with yellow)

However, in the method signature of extracted method generated by GPT-4o, the type for domainAxisLocations was a list of AxisLocation. When replacing the cloned codes in the original method, the method call caused compilation errors because of the mismatch of types.

The number suggests that refactoring within the same file is more stable, while refactoring clones in different files caused more compilation errors for GPT-4o.

Correctness. Another important measure of GPT-4o's refactoring effectiveness is the number of test failures, as these directly reflect whether the refactored code maintains its original functionality. If we look at scenarios where clones are within the same file, GPT-4o performs quite well overall, with only a small number of test failures—just 2 failures for Type-2 whole method clones, and slightly higher numbers for Type-3 clones, specifically 1 failure for whole methods and 3 failures for fragment clones. However, when clones are in different files, the situation becomes more challenging. GPT-4o significantly struggles here, especially for whole method Type-2 clones, where test failures increase sharply to 55. Similarly, fragment clones across different files cause 11 test failures for Type-2 clones. Interestingly, Type-3 clones in cross-file scenarios have fewer test failures compared to Type-2: 82 failures for whole method clones and only 4 failures for fragment clones.

Overall, the results indicate that GPT-4o is effective at preserving functional correctness when refactoring within the same file. On the other hand, cross-file refactoring presents more difficulties, leading to a higher rate of test fails. Although JDeodorant marked some clones as non-refactorable due to precondition

violations, GPT-4o was able to overcome these violations and successfully refactor the clones.

Interestingly, we found 248 cases where GPT-4o was able to correctly refactor code clones, even though the clone types it identified were incorrect. This suggests that GPT-4o does not necessarily rely on an accurate classification of clone types in order to generate refactored code. In other words, even when the model misunderstands the structural similarity between code fragments, it can still recognize shared logic and apply correct refactoring. This highlights a key strength of LLMs: their ability to reason beyond classifications.

Lambda expression. Lambda expression was first introduced in Java 8, and it can be used to support functional programming. For code refactoring, traditional parameterization may introduce side effects or break data dependencies, lambda expressions allow behavioral differences to be passed as arguments and executed in their original text [86]. In this part, we compared three scenarios: (1) JDeodorant used lambda expressions while GPT-4o did not, (2) GPT-4o used lambda expressions while JDeodorant did not, and (3) both tools used lambda expressions.

Case 1, JDeodorant used lambda expression while GPT-4o used the conventional approach.

```
private java.util.List refreshTicksExtracted(Graphics2D g2, Rectangle2D dataArea,
     RectangleEdge edge,
                     Function<Rectangle2D, Double> arg0 ,
                          Function<Rectangle2D, Double> arg1 , TextAnchor arg2,
                         RectangleEdge arg3,
                    double arg4, double arg5, TextAnchor arg6, TextAnchor arg7) {
             List ticks = new java.util.ArrayList();
             Font tickLabelFont = getTickLabelFont();
             g2.setFont(tickLabelFont);
             double size = getTickUnit().getSize();
             int count = calculateVisibleTickCount();
             double lowestTickValue = calculateLowestVisibleTickValue();
             double previousDrawnTickLabelPos = 0.0;
             double previousDrawnTickLabelLength = 0.0;
       ......
```

Listing 10: Comparison case 1 - lambda expression by JDeodorant (highlighted with yellow)

```
private List refreshTicks(Graphics2D g2, Rectangle2D dataArea, RectangleEdge edge, boolean
     isHorizontal) {
   List ticks = new java.util.ArrayList();

   Font tickLabelFont = getTickLabelFont();
   g2.setFont(tickLabelFont);

   double size = getTickUnit().getSize();
   int count = calculateVisibleTickCount();
   double lowestTickValue = calculateLowestVisibleTickValue();

   double previousDrawnTickLabelPos = 0.0;
   double previousDrawnTickLabelLength = 0.0;
   ......
}
```

Listing 11: Comparison case 1 - conventional approach by GPT-4o

As we can see from Listing 10 and Listing 11, JDeodorant and GPT-4o adopted different strategies for refactoring the methods refreshTicksHorizontal and refreshTicksVertical, which were Type-2 clones. JDeodorant employed lambda expressions and introduced eight separate parameters using functional interfaces. This approach provides flexibility and high abstraction, which allows developers to handle different scenarios by passing different implementations. On the other hand, GPT-4o simplified the refactoring by using a single boolean parameter. This simplified version improved readability and was also easy to understand, while still preserving the original functionality. Both methods have their strengths: JDeodorant stood out for its flexibility in accommodating future changes, while GPT-4o prioritized simplicity and clarity. This made the resulting code easier to understand and maintain, especially for developers who might not be as comfortable with lambda expressions.

Case 2, JDeodorant used conventional approach while GPT-4o used lambda expression.

```
protected boolean getItemVisibleExtracted(Boolean arg0, Boolean arg1, boolean arg2) {
              Boolean flag = arg0;
              if (flag == null) {
                     flag = arg1;
              }
              if (flag != null) {
                     return flag.booleanValue();
              } else {
                     return arg2;
              }
       }

#### In `LineAndShapeRenderer.java` (Replacing `getItemLineVisible`)
return getItemVisibleExtracted(this.linesVisible, getSeriesLinesVisible(series),
     this.baseLinesVisible);

#### In `XYLineAndShapeRenderer.java` (Replacing `getItemShapeVisible`)
return getItemVisibleExtracted(this.shapesVisible, getSeriesShapesVisible(series),
     this.baseShapesVisible);
```

Listing 12: Comparison 2 - conventional approach for clone group 1138-1-4 by JDeodorant

```
protected boolean getItemVisibility(int series, int item, Boolean globalFlag,
 java.util.function.Function<Integer, Boolean> seriesFlagGetter, boolean baseFlag) {
   Boolean flag = globalFlag;
   if (flag == null) {
      flag = seriesFlagGetter.apply(series);
   }
   if (flag != null) {
      return flag.booleanValue();
   } else {
      return baseFlag;
   }
}

#### In `LineAndShapeRenderer.java` (Replacing `getItemLineVisible`)
return getItemVisibility(series, item, this.linesVisible,
                         this::getSeriesLinesVisible, this.baseLinesVisible);

#### In `XYLineAndShapeRenderer.java` (Replacing `getItemShapeVisible`)
return getItemVisibility(series, item, this.shapesVisible,
                         this::getSeriesShapesVisible, this.baseShapesVisible);
```

Listing 13: Comparison 2 - lambda expression for clone group 1138-1-4 by GPT-4o (highlighted with yellow)

Listing 12 and Listing 13 shows the conventional approach JDeodorant used and lambda expression GPT-4o used to refactor clone group 1138-1-4. JDeodorant took 3 boolean parameters, which simplified the method calls. On the other hand, GPT-4o introduced lambda expression, which is flexible way as it allows different behaviors to be passed.

Case 3, Both JDeodorant and GPT-4o used lambda expression (see replication package [15]).

When refactoring clone group 981-2-4, both JDeodorant and GPT-4o employed lambda expressions, but each approached the extraction differently. JDeodorant extracted a method with multiple functional parameters, providing flexibility by passing behavior as arguments. The method calls included nested lambda expressions and explicit type annotations. On the other hand, GPT-4o's refactoring introduced a boolean parameter and utilized simpler lambda expressions. Both methods effectively addressed duplication, each offering its own style of abstraction.

Observation: ***Unuseful method extraction by GPT.*** During our analysis of refactoring code fragment clones, we observed several cases where GPT-4o did not effectively refactor the entire identified code clones. Specifically, we discovered 62 distinct cases where GPT-4o's refactoring was incomplete, leaving significant portions of the original cloned fragments untouched. Among these incomplete refactorings were particularly superficial instances, where the refactoring only involved minor adjustments, sometimes merely altering or moving a single line of code. Such minimal modifications failed to meaningfully address the clones and did not contribute towards the broader goal of improving code quality and maintainability. As a result, we classified all 61 of these instances as unhelpful refactorings, given their limited impact on actual code enhancement.

In addition to these refactorings, we encountered another category of ineffective refactoring efforts by GPT-4o. Specifically, there were instances where GPT-4o was unable to accurately identify or extract the common logic shared across cloned fragments. In this case, GPT-4o simply relocated the cloned code fragments into the extracted method without consolidating the common logic. To manage the differences between clones, GPT-4o inserted conditional statements within the extracted method. This approach resulted in the persistence of duplicate logic, although in a slightly restructured form. Overall, we found 29 cases, each still containing clones despite the supposed refactoring. Such outcomes diminish the effectiveness and benefit of refactoring, as they fail to achieve the primary goal of reducing duplication.

Observation: ***Excessive refactoring beyond clone boundaries.*** Despite the cases of ineffective method extraction, we also noted an opposite phenomenon, where GPT-4o refactored beyond the boundaries of the identified cloned fragments. Specifically, we identified 82 instances in which GPT-4o's refactoring involved more lines than the original cloned fragments. In exces-

sive refactoring, we found two cases; the first is expanded refactoring, where GPT-4o refactored more lines than just the clones. The second case is comprehensive refactoring, where GPT-4o extracted the entire method. While this kind of refactoring can improve parts of the code, making too many changes at once can be risky. It might accidentally change parts of the logic that were not supposed to be changed, making the code more complicated and harder to understand. It could even introduce new bugs in certain cases.

RQ2 Summary

Our findings reveal several limitations in GPT-4o's refactoring capabilities. While it performs reasonably well on clones within the same file, its effectiveness drops sharply when handling clones across different files, especially for Type-2 and Type-3 clones. Compilation errors and test failures were significantly more common in these cross-file scenarios. Additionally, GPT-4o produced 55 cases of unhelpful refactorings, where clone extraction was either superficial or incomplete. And 28 cases were found where the refactoring failed to remove duplication. We also observed 82 instances of excessive refactoring that extended beyond the clone boundaries, introducing potential risks to code stability and maintainability. On the use of lambda expressions, GPT-4o showed flexibility, offering simpler, more readable which is different with JDeodorant's more abstract but extensible implementations. These findings highlight GPT-4o's potential in clone refactoring, but also its current limitations in reliably producing high-quality and behavior-preserving transformations.

Table 5: Summary of clone detection and refactoring performance for traditional tools and LLMs

Tools/Models	**Clones Detected**	**Refactoring**								
		Compiled	Test passed	Correct refactoring	Incomplete refactoring	Direct clone removal	Post-extraction clone deletion	Call-based clone substitution	Fragmentation of logic	Clone removel by delegation
JDeodorant	20	20	20	20	0	0	0	0	0	0
AntiCopyPaster	20	20	20	20	0	0	0	0	0	0
Gemini 2.5 Pro	20	20	20	3	4	3	10	0	0	0
GPT-3.5	0	0	0	0	0	0	0	0	0	0
GPT-4o	20	20	20	14	6	0	0	0	0	0
GPT-5	20	20	20	15	3	0	0	1	0	1
DeepSeek-R1	20	20	20	7	9	0	0	2	2	0
Claude Sonnet 3.7	20	20	20	11	6	1	0	1	1	0
Claude Opus 4.1	20	20	20	10	2	2	3	2	1	0

Note: Direct clone removal refers to cases where LLMs eliminated clones not by extracting clones into a new method, but by directly deleting one of the cloned methods. **Post-extraction clone deletion** means extracting the clones into a new method and replacing one of the cloned methods with a call to that method, while simply deleting another cloned method entirely. **Call-based clone substitution** describes situations where the entire method, not just the cloned fragments, was refactored. **Fragmentation of logic** means creating some new methods from the original method, then simply deleted the cloned method. **Clone removal by delegation** refers to cases where models refactor the original method to use an extracted helper method, and replace cloned method with call to the original method.

Table 6: Frequency of each incorrect refactoring

Categories	Subcategory	Gemini 2.5 Pro	GPT-4o	GPT-5	DeepSeek-R1	Claude Sonnet 3.7	Claude Opus 4.1
Refactoring Effectiveness	Incomplete Refactoring	20%	30%	15%	45%	30%	10%
Clone Removal Strategies	Post-Extraction Clone Deletion	50%	0%	0%	0%	0%	15%
	Direct Clone Removal	15%	0%	0%	0%	5%	10%
	Call-Based Clone Substitution	0%	0%	5%	10%	5%	10%
	Fragmentation of Logic	0%	0%	0%	10%	5%	5%
	Clone Removal by Delegation	0%	0%	5%	0%	0%	0%

Note: The percentage is calculated as the number of cases in each subcategory divided by the total of 20 cases.

Table 7: Definition of incorrect refactoring categories

Category	Definition
Incomplete refactoring	The extracted method either does not fully cover all the cloned code or still contains cloned segments.
Expanded refactoring	Refactoring includes additional lines beyond the original clone.
Comprehensive refactoring	The entire method, not just the cloned fragments, is refactored
Post-extraction clone deletion	Extracting the clones into a new method and replacing one of the cloned methods with a call to that method, while simply deleting another cloned method entirely
Direct clone removel	Eliminating clones not by extracting clones into a new method, but by directly deleting one of the cloned methods.
Call-based clone substitution	Eliminating clones not by extracting clones into a new method, but by directly calling another cloned methods.
Fragmentation of logic	Creating some new methods from the original method, then simply delete the cloned method.
Clone removal by delegation	Refactoring the original method to use an extracted helper method, and replace cloned method with call to the original method.

4.3. Case Study

For each tool, we examined its ability to identify clones, apply the "Extract Method" refactoring correctly, preserve code functionality, and avoid compilation errors.

4.3.1. Clone Identification

The results in Table 5 and Table 6 show the performance in clone detection among the tools and models evaluated. JDeodorant, AntiCopyPaster, Gemini 2.5 Pro, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1 all successfully detected all 20 code clones, demonstrating strong performance in identifying duplicated code. In contrast, GPT-3.5 failed to detect any clones, indicating its limitation in recognizing duplicated code.

We observed two types of failed responses from GPT-3.5 in its clone identification. The following section discusses these cases in detail.

Observation:*Unrelated output* GPT-3.5 produced unrelated codes in 18 of the 20 cases, although the instruction was asked to detect the clones in the file. For instance, in the file CompositeFigureCreationTool.java, there were two cloned methods, mouseDown and mouseDown_Cloned. The prompt we used for clone detection asked the model to return either 'clones found' or 'no clones found.'

```
public class CompositeFigureCreationTool extends CreationTool {
    public void mouseMove(MouseEvent e, int x, int y) {
        if ((getContainerFigure() != null) &&
        !getContainerFigure().containsPoint(e.getX(), e.getY())) {
         //here you might want to constrain the mouse movements to the size of the
```

Listing 14: Unrelated output example

However, as shown in Listing 14, the model ignored the instruction and generated unrelated code from the file, which was neither related to the task nor meaningful in context. This indicated the limited capacity of GPT-3.5.

Observation:*Incorrect output* In addition to the unrelated outputs, we also identified 2 out of 20 cases where GPT-3.5 failed to detect clones in the file.

```
public void animationStep() {
    FigureEnumeration fe = figures();
    while (fe.hasNextFigure())
        Figure f = fe.nextFigure();

        if(!(f instanceof ConnectionFigure))
            ((AnimationDecorator) f).animationStep();
        }
   }
}

public void animationStep_Cloned() {
    FigureEnumeration fe = figures();
   while (fe.hasNextFigure())
    Figure f = fe.nextFigure();

        if(!(f instanceof ConnectionFigure))
            ((AnimationDecorator) f).animationStep();
```

Listing 15: Example clones (highlighted with yellow) of the incorrect output

As shown in Listing 15, method animationStep and method animationStep_Cloned from BouncingDrawing.java were Type-1(c) clones which only differ in layout. However, the model incorrectly reported 'no clones found'. This further highlights GPT-3.5's limits to recognize clones.

4.3.2. Clone Refactoring

When it comes to clone refactoring, both JDeodorant and AntiCopyPaster performed perfectly. They refactored them correctly without any issues. Their refactoring compiled successfully, passed all tests, and the extractions were all correct. GPT-4o also did a good job, with 14 out of 20 refactorings done correctly, although 6 were incomplete. In contrast, Gemini 2.5 Pro struggled more. Only 3 of its refactorings were correct, and it encountered cases of incomplete refactoring, deleting the cloned method after extraction or directly deleting the cloned method. This suggests that it had trouble applying proper refactoring techniques. GPT-3.5 did not refactor anything, as it could not detect any clones to begin with. Claude Sonnet 3.7 successfully performed a refactoring for 11 instances, which were more than Gemini 2.5 Pro and DeepSeek-R1, but fewer than GPT-4o. For the other 9 cases, 6 were incomplete refactorings. There was also one case of directly deleting the cloned method, one case of directly replacing the cloned method with a method call to the original method, and one case of creating multiple new methods. GPT-5 achieved the best overall

performance among the LLMs, with 15 out of 20 refactorings done correctly and 3 incomplete, showing more stability and fewer destructive behaviors than the other models. Claude Opus 4.1 performed 10 correct refactorings, while the remaining cases included 2 incomplete refactorings and several problematic patterns such as direct deletion, post-extraction clone deletion, call-based substitution, and fragmented logic.

We found new several patterns except incomplete refactoring of how models removed clones after detection incorrectly. We identify the following strategies observed in our case studies:

Observation: ***Post-extraction clone deletion.*** We identified 10 cases for Gemini 2.5 pro and 3 cases for Claude Opus 4.1 where they performed refactoring by extracting the clones into a new method and replacing one of the cloned methods with a call to that method, while simply deleting another cloned method entirely, as defined in Table 7. This behavior raised concern because it can result in the loss of a potentially meaningful method. For instance, in the file Helper.java, there were two cloned methods getDrawingView and getDrawingView_clone. They were Type-2(b) clones, which differ in identifiers. After LLM's refactoring, as shown in Listing 16,

```
static public DrawingView findDrawingViewInChildren(Container container) {
    Component[] components = container.getComponents();
    for (Component component : components) {
        if (component instanceof DrawingView) {
            return (DrawingView) component;
        } else if (component instanceof Container) {
            DrawingView foundView = findDrawingViewInChildren((Container) component);
            if (foundView != null) {
                return foundView;
            }
        }
    }
    return null;
}

- static public DrawingViewgetDrawingView_Cloned(Container container) {
- DrawingView o = null;
- Component[] c = container.getComponents();
- for (int ij = 0; ij < c.length; ij++) {
- if (c[ij] instanceof DrawingView) {
- return (DrawingView)c[ij];
- }
- else if (c[ij] instanceof Container) {
- o = getDrawingView((Container)c[ij]);
- if (o != null) {
- return o;
- }
- }
- }
- return null;
```

```
31 - }
32
33 static public DrawingView getDrawingView(Container container) {
34     return findDrawingViewInChildren(container);
35 }
```

Listing 16: Gemini 2.5 Pro post-extraction clone deletion example

From lines 1 - 14, the duplicated code was extracted into a new method named findDrawingViewInChildren. The method getDrawingView was refactored to simply call the new method. However, another cloned method getDrawingView_clone was removed entirely.

Observation: ***Direct clone removal.*** As demonstrated in Table 7, we found 6 cases in total where LLMs eliminated clones not by extracting clones into a new method, but by directly deleting one of the cloned methods.

```
1  static public int direction(int x1, int y1, int x2, int y2) {
2      int direction = 0;
3      int vx = x2 - x1;
4      int vy = y2 - y1;
5      if (vy < vx && vx > -vy) {
6          direction = EAST;
7      }
8      else if (vy > vx && vy > -vx) {
9          direction = NORTH;
10     }
11     else if (vx < vy && vx < -vy) {
12         direction = WEST;
13     }
14     else {
15         direction = SOUTH;
16     }
17     return direction;
18 }
19 - static public int direction_Cloned(int x1, int y1, int x2, int y2) {
20 - int d = 0;
21 - int x = x2 - x1;
22 - int y = y2 - y1;
23 - if (y < x && x > -y) {
24 - d = EAST;
25 - }
26 - else if (y > x && y > -x) {
27 - d = NORTH;
28 - }
29 - else if (x < y && x < -y) {
30 - d = WEST;
31 - }
32 - else {
33 - d = SOUTH;
34 - }
35 - return d;
```

```
- }
```

Listing 17: Gemini 2.5 Pro direct clone removal example

For example, in the Geom.java file, the method direction and direction_Cloned were Type-2(a) clones. When Gemini 2.5 Pro refactored them, it directly deleted the method direction_Cloned, instead of creating an extracted method and replacing them with method calls. As we can see from Listing 17, before refactoring there were methods direction and direction_Cloned. However, after refactoring, the direction_Cloned was deleted, leaving direction unchanged.

Observation: ***Call-based clone substitution.*** We also defined call-based clone substitution in Table 7. In 6 cases, we observed that GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1 did not perform method extraction. Instead, it directly replaced the cloned method by calling the original method.

```
1  public void init() {
2      createIconkit();
3      getVersionControlStrategy().assertCompatibleVersion();
4      setUndoManager(new UndoManager());
5
6      getContentPane().setLayout(new BorderLayout());
7
8      fView = createDrawingView();
9
10     JPanel attributes = createAttributesPanel();
11     createAttributeChoices(attributes);
12     getContentPane().add("North", attributes);
13
14     JPanel toolPanel = createToolPalette();
15     createTools(toolPanel);
16     getContentPane().add("West", toolPanel);
17
18     getContentPane().add("Center", (Component)view());
19     JPanel buttonPalette = createButtonPanel();
20     createButtons(buttonPalette);
21     getContentPane().add("South", buttonPalette);
22
23     initDrawing();
24     // JFC should have its own internal double buffering...
25     //setBufferedDisplayUpdate();
26     setupAttributes();
27 }
28
29 public void init_Cloned() {
30         init();
31 }
```

Listing 18: DeepSeek-R1 call-based clone substitution example (highlighted with yellow)

For example, as shown in Listing 18, in the case of refactoring the init method in DrawApplet.java, instead of extracting a new method, DeepSeek-R1 replaced the cloned method with a simple method call to the original init() method in line 30.

Observation: ***Fragmentation of logic.*** The definition of fragmentation of logic is provided in Table 7. Regarding the refactoring performed by DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1, we observed 4 cases in total that models created some new methods from the original method, then simply deleted

the cloned method. For example, when refactoring method buildChildMenus in CTXWindowMenu.java, as shown in the Listing 19,

```
1  void buildChildMenus() {
2      JInternalFrame[] array = desktop.getAllFrames();
3      updateWindowCommands(array.length > 0);
4
5      if (array.length == 0) {
6          return;
7      }
8
9      addSeparator();
10     createChildMenuItems(array);
11 }
12
13 + private void updateWindowCommands(boolean enabled) {
14 + cascadeCommand.setEnabled(enabled);
15 + tileHCommand.setEnabled(enabled);
16 + tileVCommand.setEnabled(enabled);
17 + arrangeHCommand.setEnabled(enabled);
18 + arrangeVCommand.setEnabled(enabled);
19 + }
20 + private void createChildMenuItems(JInternalFrame[] frames) {
21 + for (int i = 0; i < frames.length; i++) {
22 + ChildMenuItem menu = new ChildMenuItem(frames[i]);
23 + menu.setState(i == 0);
24 + menu.addActionListener(createChildMenuItemListener());
25 + menu.setIcon(frames[i].getFrameIcon());
26 + add(menu);
27 + }
28 + }
```

Listing 19: DeepSeek-R1 fragmentation of logic example

In lines 13 - 28, DeekSeek-R1 created two new methods: updateWindowCommands and createChildMenuItems from the original method buildChildMenus. Then it replaced the corresponding code fragments with method calls. After this replacement, the cloned method was also deleted.

Observation: ***Clone removal by delegation.*** We also observed 1 case from GPT-5 where the model extracted the method, but it only replaced original method with the extracted method call. For the cloned method, instead it replaced with the original method call.

```
1 public void rotate(double angle) {
2     willChange();
3     fRotation = nearestRotationIndex(angle);
4     changed();
5 }
6
7 public void rotate_Cloned(double angle) {
```

```
8       rotate(angle);
9   }
10
11  private int nearestRotationIndex(double angle) {
12      double dist = Double.MAX_VALUE;
13      int best = 0;
14      for (int i = 0; i < rotations.length; ++i) {
15          double d = Math.abs(angle - rotations[i]);
16          if (d < dist) {
17              dist = d;
18              best = i;
19          }
20      }
21      return best;
22  }
```

Listing 20: GPT-5 clone removal by delegation example

In Listing 20, the methods rotate and rotate_Cloned were identified as Type-2 clones. When GPT-5 performed the refactoring, lines 2-4 of the original method body were correctly replaced with a call to the extracted method. However, in line 8, the cloned method was modified to call the original method rather than the extracted method.

RQ3 Summary

Overall, we found that LLMs vary in their consistency compared to traditional tools when it comes to clone identification and refactoring. All LLMs except GPT-3.5 successfully identified all clones, matching the performance of JDeodorant and AntiCopyPaster in detection. However, their refactoring quality differed widely. GPT-5 showed the strongest consistency among LLMs, with 15 out of 20 correct refactorings and fewer destructive behaviors. GPT-4o also performed relatively well (14/20 correct), though it exhibited several incomplete refactorings. In contrast, Gemini 2.5 Pro and DeepSeek-R1 often produced unstable or destructive refactorings, such as post-extraction clone deletion, direct method removal, and fragmented logic. Claude models showed mixed results: Claude Sonnet 3.7 achieved 11 correct refactorings, while Claude Opus 4.1 achieved 10, but both also displayed problematic patterns. Traditional tools (JDeodorant and AntiCopyPaster) were perfectly consistent, with 100% correct and compilable refactorings.
These findings highlight that while LLMs can match traditional tools in clone detection, their refactoring consistency is far less reliable, with performance varying significantly across models and often introducing non-standard or destructive behaviors.

4.4. Statistical Analysis

As outlined in Section 3.9, we applied the McNemar's test. In this subsection, we present and discuss the findings derived from the test.

Table 8: McNemar's test results comparing GPT-4o and JDeodorant on clone classification across different clone types and settings.

Scenario	χ^2	p-value
Type-1 Code Fragment (Different File)	0.00	1.00
Type-1 Code Fragment (Same File)	0.00	1.00
Type-1 Whole Method (Different File)	0.00	1.00
Type-2 Code Fragment (Different File)	133.01	9.01e-31
Type-2 Code Fragment (Same File)	126.01	3.06e-29
Type-2 Whole Method (Different File)	138.01	7.26e-32
Type-2 Whole Method (Same File)	94.01	3.14e-22
Type-3 Code Fragment (Different File)	22.04	2.67e-06
Type-3 Code Fragment (Same File)	6.13	1.33e-02
Type-3 Whole Method (Different File)	17.05	3.64e-05
Type-3 Whole Method (Same File)	3.20	7.36e-02

4.4.1. RQ1: How well can LLMs classify code clones?

Table 8 summarizes the McNemar's test results comparing GPT-4o and JDeodorant on clone classification. For Type-1 clones, all p-values are equal to 1.0, indicating no significant difference between the two tools. In contrast, for Type-2 clones, the p-values are extremely small with all p-values $< 10^{-29}$, which shows strong evidence that GPT-4o and JDeodorant behave differently on these cases. For Type-3 clones, most scenarios show statistically significant differences, but the effects are less pronounced than for Type-2. The only exception is Type-3 whole method in the same file where the p-value of 0.074 indicates no significant difference. Overall, these results suggest that while GPT-4o and JDeodorant are comparable on Type-1 clones, their behavior diverges considerably for Type-2 and to a lesser extent, Type-3 clones.

Table 9: McNemar's test comparing GPT-4o against JDeodorant across compilation, test-pass, and refactoring metrics.

Dataset	Compilation	Test pass	Correct refactor
Type-1 code fragment different files	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$
Type-1 code fragment same file	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$
Type-2 code fragment different files	$\chi^2 = 133.01,\ p = 9.01 \times 10^{-31}$	$\chi^2 = 126.01,\ p = 3.06 \times 10^{-29}$	$\chi^2 = 138.01,\ p = 7.26 \times 10^{-32}$
Type-2 code fragment same file	$\chi^2 = 126.01,\ p = 3.06 \times 10^{-29}$	$\chi^2 = 126.01,\ p = 3.06 \times 10^{-29}$	$\chi^2 = 126.01,\ p = 3.06 \times 10^{-29}$
Type-2 whole method different files	$\chi^2 = 138.01,\ p = 7.26 \times 10^{-32}$	$\chi^2 = 138.01,\ p = 7.26 \times 10^{-32}$	$\chi^2 = 138.01,\ p = 7.26 \times 10^{-32}$
Type-2 whole method same file	$\chi^2 = 94.01,\ p = 3.14 \times 10^{-22}$	$\chi^2 = 94.01,\ p = 3.14 \times 10^{-22}$	$\chi^2 = 94.01,\ p = 3.14 \times 10^{-22}$
Type-3 code fragment different files	$\chi^2 = 22.04,\ p = 2.67 \times 10^{-6}$	$\chi^2 = 22.04,\ p = 2.67 \times 10^{-6}$	$\chi^2 = 22.04,\ p = 2.67 \times 10^{-6}$
Type-3 code fragment same file	$\chi^2 = 6.13,\ p = 0.013$	$\chi^2 = 6.13,\ p = 0.013$	$\chi^2 = 6.13,\ p = 0.013$
Type-3 whole method different files	$\chi^2 = 17.05,\ p = 3.64 \times 10^{-5}$	$\chi^2 = 17.05,\ p = 3.64 \times 10^{-5}$	$\chi^2 = 17.05,\ p = 3.64 \times 10^{-5}$
Type-3 whole method same file	$\chi^2 = 3.20,\ p = 0.074$	$\chi^2 = 3.20,\ p = 0.074$	$\chi^2 = 3.20,\ p = 0.074$
Type-1 whole method different files	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$	$\chi^2 = 0.00,\ p = 1.000$

4.4.2. RQ2: How well can LLMs refactor code clones?

We conducted McNemar's tests across different clone types and scenarios, examining three metrics: compilation success, test pass rate, and refactoring correctness. The results are presented in Table 9.

The analysis reveals distinct performance patterns across clone types. For Type-1 clones, both tools showed no significant differences across all metrics, regardless of whether the clones appeared in different files or the same file ($\chi^2 = 0.00$, $p = 1.000$ for all comparisons). This indicates comparable performance on straightforward clone detection and refactoring tasks.

In contrast, Type-2 clones demonstrated highly significant differences between the two tools. For code fragments in different files, significant differ-

ences were observed in compilation ($\chi^2 = 133.01$, $p = 9.01 \times 10^{-31}$), test pass ($\chi^2 = 126.01$, $p = 3.06 \times 10^{-29}$), and refactoring correctness ($\chi^2 = 138.01$, $p = 7.26 \times 10^{-32}$). Similar patterns emerged for code fragments in the same file and whole methods, with all comparisons showing $p < 0.001$. The consistently large chi-squared values across Type-2 datasets suggest substantial performance divergence between the tools on this clone type.

Type-3 clones exhibited varying degrees of difference. Code fragments in different files showed significant differences across all metrics ($\chi^2 = 22.04$, $p = 2.67 \times 10^{-6}$), while code fragments in the same file displayed smaller but still significant differences ($\chi^2 = 6.13$, $p = 0.013$). For whole methods, different files showed significant differences ($\chi^2 = 17.05$, $p = 3.64 \times 10^{-5}$), but methods in the same file showed no significant difference ($\chi^2 = 3.20$, $p = 0.074$). These results suggest that the tools' relative performance on Type-3 clones depends on both granularity and file location.

Table 10: Pairwise McNemar's tests on the correct refactoring of case studies.

Model A	Model B	χ^2	p
GPT-5	Claude Sonnet 3.7	2.667	0.2188
GPT-5	Claude Opus 4.1	3.571	0.1250
GPT-5	JDeodorant	5.000	0.0625
GPT-5	Gemini 2.5 Pro	10.286	0.0018
GPT-5	AntiCopyPaster	5.000	0.0625
GPT-5	DeepSeek-R1	6.400	0.0215
GPT-5	GPT-4o	0.143	1.0000
Claude Sonnet 3.7	Claude Opus 4.1	0.143	1.0000
Claude Sonnet 3.7	JDeodorant	9.000	0.0039
Claude Sonnet 3.7	Gemini 2.5 Pro	8.000	0.0078
Claude Sonnet 3.7	AntiCopyPaster	9.000	0.0039
Claude Sonnet 3.7	DeepSeek-R1	2.667	0.2188
Claude Sonnet 3.7	GPT-4o	3.000	0.2500
Claude Opus 4.1	JDeodorant	10.000	0.0020
Claude Opus 4.1	Gemini 2.5 Pro	4.455	0.0654
Claude Opus 4.1	AntiCopyPaster	10.000	0.0020
Claude Opus 4.1	DeepSeek-R1	0.818	0.5488
Claude Opus 4.1	GPT-4o	2.000	0.2891
JDeodorant	Gemini 2.5 Pro	17.000	0.0000
JDeodorant	AntiCopyPaster	0.000	1.0000
JDeodorant	DeepSeek-R1	13.000	0.0002
JDeodorant	GPT-4o	6.000	0.0312
Gemini 2.5 Pro	AntiCopyPaster	17.000	0.0000
Gemini 2.5 Pro	DeepSeek-R1	2.000	0.2891
Gemini 2.5 Pro	GPT-4o	11.000	0.0010
AntiCopyPaster	DeepSeek-R1	13.000	0.0002
AntiCopyPaster	GPT-4o	6.000	0.0312
DeepSeek-R1	GPT-4o	5.444	0.0391

4.4.3. RQ3: How consistent are LLMs' clone identification and refactoring capabilities across different models compared to traditional approaches?

We conducted pairwise McNemar's tests to compare the performance of different models on the 20 case studies, with results presented in Table 10. Because

all models except GPT-3.5 performed exactly the same in clone identification, compilation success and passing test cases, we only analyzed correct refactoring. The test examines whether two models show significantly different performance by analyzing cases where their predictions disagree.

Several model pairs exhibited significant performance differences. GPT-5 showed significant differences when compared with Gemini 2.5 Pro ($\chi^2 = 10.286$, $p = 0.0018$) and DeepSeek-R1 ($\chi^2 = 6.400$, $p = 0.0215$). Claude Sonnet 3.7 demonstrated significant differences from JDeodorant ($\chi^2 = 9.000$, $p = 0.0039$), Gemini 2.5 Pro ($\chi^2 = 8.000$, $p = 0.0078$), and AntiCopyPaster ($\chi^2 = 9.000$, $p = 0.0039$). Claude Opus 4.1 showed significant differences when compared to JDeodorant ($\chi^2 = 10.000$, $p = 0.0020$) and AntiCopyPaster ($\chi^2 = 10.000$, $p = 0.0020$). Notable differences were also observed between JDeodorant and Gemini 2.5 Pro ($\chi^2 = 17.000$, $p < 0.0001$), JDeodorant and DeepSeek-R1 ($\chi^2 = 13.000$, $p = 0.0002$), Gemini 2.5 Pro and AntiCopyPaster ($\chi^2 = 17.000$, $p < 0.0001$), Gemini 2.5 Pro and GPT-4o ($\chi^2 = 11.000$, $p = 0.0010$), as well as AntiCopyPaster and DeepSeek-R1 ($\chi^2 = 13.000$, $p = 0.0002$).

In contrast, many model pairs showed no significant differences in performance. GPT-5 and Claude Sonnet 3.7 ($\chi^2 = 2.667$, $p = 0.2188$), GPT-5 and GPT-4o ($\chi^2 = 0.143$, $p = 1.0000$), and Claude Sonnet 3.7 and Claude Opus 4.1 ($\chi^2 = 0.143$, $p = 1.0000$) all performed comparably. JDeodorant and AntiCopyPaster showed no difference in their predictions on discordant pairs ($\chi^2 = 0.000$, $p = 1.0000$). These results indicate substantial heterogeneity in model performance, with some models differing significantly while others achieve similar results on these refactoring tasks.

5. Discussion

5.1. Takeaway 1: GPT-4o struggles in classifying Type-2 clones

As shown in Section 4, GPT-4o correctly classified all Type-1 clones. For Type-3 clones, it can still achieve 72% to 91.4% for different scenarios. However, when it comes to Type-2 clones, the performance dropped significantly, with the highest accuracy only 56.6%. We discussed the possible root cause of such failure in the following paragraphs.

Consider the code fragments shown in Listing 21. These two fragments represent a typical Type-2 clone where the algorithm remains identical but identifiers have been systematically renamed (e.g., axis → domainAxis, savedComposite → originalComposite). Additionally, the order of conditional branches (HORIZONTAL versus VERTICAL) has been swapped. Despite these superficial differences, the statement-level structure is perfectly preserved. Every statement in Fragment A has a direct counterpart in Fragment B with no additions, deletions, or functional modifications.

```
// Fragment A
else if (marker instanceof IntervalMarker) {
    IntervalMarker im = (IntervalMarker) marker;
    double start = im.getStartValue();
    double end = im.getEndValue();
```

```
    Range range = axis.getRange();
    if (!(range.intersects(start, end))) return;
    final Composite savedComposite = g2.getComposite();
    g2.setComposite(AlphaComposite.getInstance(
        AlphaComposite.SRC_OVER, marker.getAlpha()));
    double start2d = axis.valueToJava2D(start, dataArea,
        plot.getRangeAxisEdge());
    // ... [middle section omitted for brevity]
    if (orientation == PlotOrientation.HORIZONTAL) {
        low = Math.max(low, dataArea.getMinX());
        high = Math.min(high, dataArea.getMaxX());
        rect = new Rectangle2D.Double(low, dataArea.getMinY(),
            high - low, dataArea.getHeight());
    } else if (orientation == PlotOrientation.VERTICAL) {
        low = Math.max(low, dataArea.getMinY());
        high = Math.min(high, dataArea.getMaxY());
        rect = new Rectangle2D.Double(dataArea.getMinX(), low,
            dataArea.getWidth(), high - low);
    }
    // ... [remaining code]
    g2.setComposite(savedComposite);
}

// Fragment B
else if (marker instanceof IntervalMarker) {
    IntervalMarker im = (IntervalMarker) marker;
    double start = im.getStartValue();
    double end = im.getEndValue();
    Range range = domainAxis.getRange();
    if (!(range.intersects(start, end))) return;
    final Composite originalComposite = g2.getComposite();
    g2.setComposite(AlphaComposite.getInstance(
        AlphaComposite.SRC_OVER, marker.getAlpha()));
    double start2d = domainAxis.valueToJava2D(start, dataArea,
        plot.getDomainAxisEdge());
    // ... [middle section omitted for brevity]
    if (orientation == PlotOrientation.VERTICAL) {
        low = Math.max(low, dataArea.getMinX());
        high = Math.min(high, dataArea.getMaxX());
        rect = new Rectangle2D.Double(low, dataArea.getMinY(),
            high - low, dataArea.getHeight());
    } else if (orientation == PlotOrientation.HORIZONTAL) {
        low = Math.max(low, dataArea.getMinY());
        high = Math.min(high, dataArea.getMaxY());
        rect = new Rectangle2D.Double(dataArea.getMinX(), low,
            dataArea.getWidth(), high - low);
    }
    // ... [remaining code]
    g2.setComposite(originalComposite);
}
```

Listing 21: Type-2 clone example with systematic identifier renaming (difference highlighted with yellow)

GPT-4o incorrectly classified this example as Type-3, exposing key weaknesses in how language models perform structural code analysis. The main problem is semantic interference: the model understands that getRangeAxisEdge()

and getDomainAxisEdge() represent different concepts (range axis versus domain axis), so it treats them as functional changes rather than renamed identifiers. While this semantic awareness is usually helpful, it becomes problematic when clone detection requires strict syntactic comparison. The swapped order of conditional branches also confuses the model. Traditional AST-based tools recognize that reordering mutually exclusive branches does not change the statement set, but GPT-4o lacks this formal reasoning capability. This example shows that despite their strong pattern recognition abilities, LLMs struggle with tasks that demand rigorous adherence to formal definitions, especially when semantic understanding contradicts syntactic requirements.

5.2. Takeaway 2: GPT refactoring despite JDeodorant precondition violations

The refactorings are not applicable for certain code clones due to precondition violations for JDeodorant. JDeodorant's refactorability analysis [21], involves a series of checks to ensure that refactoring preserves the original behavior. These preconditions cover various aspects, such as ensuring that control or data dependencies are not broken. When one or more of these preconditions are violated, JDeodorant flags the clone as non-refactorable. However, we found 250 cases, despite these precondition violations, GPT-4o can still generate refactoring suggestions for the same clones, and pass the tests.

For example, when refactoring clone ID 62-1-2 in JFreeChart, certain precondition violations, initially prevented the refactoring process. However, GPT-4o was able to successfully extract the method (see replication package [15]). In addition, the extracted method performed correctly and passed all tests.

5.3. Takeaway 3: Incomplete refactorings by extracting only part of the clone or duplicating logic in extracted methods.

As discussed in Section 4, we identified a total of 91 cases where GPT-4o did not fully perform the refactoring. Among these incomplete cases, we observed two main issues. The first issue was that GPT-4o failed to extract entire clones; instead, it included only parts of the cloned code in the extracted method, leaving the remaining portions of the clone still in the original method. See example of failing to extract entire clones in the replication package [15], two classes in clone group 309-1-2 contain Type-2 clones, which are similar code fragments with minor differences in identifiers. The two code fragments compute positions x0 and y0, then draw the line based on the orientation situation. However, when GPT-4o performed the refactoring, it only extracted the code responsible for handling the orientation conditions and drawing the lines.

The second issue occurred when GPT-4o did not properly abstract the common logic shared by two clones. Instead, it placed both clones entirely within the extracted method, using conditional statements to handle the different scenarios. For instance, in the replication package, example of failing to extract entire clones contains two code fragments clone group 1108-1-2 which are Type-2 clones, differing in identifiers. The extracted method generated by GPT-4o extracted more lines than just the clones. Furthermore, it included both clones in the method instead of consolidating the common logic.

Among those 91 incomplete refactoring cases, we found that JDeodorant did not perform refactoring on 46 instances due to precondition violations. Interestingly, GPT-4o attempted to refactor those clones by shrinking them to smaller parts. While this strategy allows GPT-4o to perform the refactoring, it often led to the incomplete refactoring mentioned above.

5.4. Takeaway 4: Excessive refactorings that go beyond the clone boundaries

During our manual inspection, we also identified cases where GPT-4o applied excessive refactoring, which introduced two issues. One issue is expanded refactorings, where refactoring introduced by GPT-4o included more lines than just the clones. The second issue is comprehensive refactorings that GPT-4o extracted entire methods than just the actual clones. First, the refactoring went beyond the duplicated lines and included additional code that was not part of the clones, which is expanded refactoring introduced in Section 4. As shown in example of expanded clone in our replication package [15], the two code fragments were Type-3 clones. However, GPT-4o's refactoring extracted a method that not only generalized the logic of cloned code but also includes additional lines that were not present in both code fragments. Code fragment 1 and code fragment 2 shared similar loop structures to compute minimum and maximum. The refactored method introduced by GPT-4o added more lines to generalize how values were defined.

The second issue is comprehensive refactoring. In these cases, GPT-4o did not just refactor the cloned code. It went further and extracted entire methods, even when the actual clones were only small fragments within those methods. For example, in clone group 503-1-2 (see example of comprehensive refactoring in replication package [15]), similar codes were found in the compareTo methods of two classes, Hour and Minute. These two code fragments were Type-2 clones, meaning they shared similar structure with minor differences, such as types and method calls. In the original code, both classes contained conditional checks on the type of the object being compared. GPT-4o correctly identified the logic between two code fragments. However, instead of extracting just the duplicated fragment, it refactored the entire compareTo method, which included surrounding code like variable declarations and return statements. As a result, code fragment clone within the method led to the full method being extracted.

5.5. Takeaway 5: All the tools besides GPT-3.5 can detect the clones

In our case study, we tested nine tools and models for their ability to detect code clones: AntiCopyPaster, JDeodorant, Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1. Among them, GPT-3.5 was the only one that consistently failed to identify the clones present in the input files. Specifically, in 18 out of 20 cases, as demonstrated in Listing 14, GPT-3.5 produced unrelated code fragments completely, disregarding the provided instructions. For the remaining two cases, it incorrectly reported 'no clones found', despite the existence of duplicated code segments. As we can see from Listing 15, although two methods were clearly Type-1 clones, GPT-3.5 still output 'no clones found'.

In contrast, the other tools and models, including AntiCopyPaster, JDeodorant, Gemini 2.5 Pro, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1 were able to detect all the clones correctly. This highlights a performance gap between GPT-3.5 and the other more advanced or specialized tools. These results suggest that GPT-3.5 requires further improvement before it can be considered reliable for clone detection tasks.

5.6. Takeaway 6: JDeodorant and AntiCopyPaster performed more stable refactoring than LLMs

In our case study, we compared the performance of JDeodorant and AntiCopyPaster with seven LLMs: Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1. With evaluation of clone refactoring across JHotDraw, the traditional tools JDeodorant and AntiCopyPaster achieved perfect results, with all "Extracted Method" refactorings applied correctly, compiling successfully, and passing tests. Among the LLMs, GPT-4o showed strong performance with 14 correct refactorings out of 20, though 6 were incomplete. GPT-5 achieved the best overall outcome, with 15 correct and 3 incomplete refactorings, and did not exhibit destructive behaviors such as direct method deletion. Claude Sonnet 3.7 ranked next with 11 correct refactorings, but also produced 6 incomplete cases, along with occasional issues such as direct method removal, call-based substitutions, and fragmented extractions. Claude Opus 4.1 performed 10 correct refactorings and 2 incomplete, but displayed more problematic behaviors, including direct deletions and post-extraction removals. DeepSeek-R1 succeeded in 7 cases, while 9 were incomplete, and it also showed unusual behaviors such as replacing cloned methods with calls to the originals or generating multiple fragmented extractions. Gemini 2.5 Pro struggled significantly, with only 3 correct refactorings and frequent incomplete refactoring results. It also involved deletion of cloned methods. By contrast, GPT-3.5 failed to perform any refactoring, as it did not reliably detect clones.

6. Implications

6.1. Implications for Practitioners

Guidance for applying LLM-based clone refactoring. The findings from RQ2 reveal conditions under which LLMs tend to perform more or less reliably in clone refactoring. It shows that LLMs perform more reliably when handling same-file clones, especially Type-1 and Type-2 cases, where duplication is localized and the probability of compilation or testing failures is relatively low. In contrast, cross-file clones present a much higher risk of incomplete elimination and compilation errors. These observations suggest that practitioners may use LLM-based refactoring primarily for same-file clones, where it can significantly accelerate clone removal, but remain cautious in cross-file or highly coupled scenarios, where static or AST-based tools remain preferable. This distinction illustrates when to use what: LLMs are best suited for localized, same-file

Type-1 or Type-2 clones, while static or AST-based approaches remain the safer option for cross-file or coupled clones. Even when applying LLM refactoring in favorable settings, outputs should be treated as candidate patches rather than production-ready code. Verifying clone elimination completeness, checking for under-extraction (i.e. incomplete refactoring) or over-extraction (i.e. expanded refactoring, comprehensive refactoring), and validating behavior preservation through tests are essential steps to integrate these methods responsibly into practice.

6.2. Implications for Researchers

Broadening evaluation criteria for LLMs. The results from RQ1 and RQ2 indicate that evaluating LLMs on clone tasks cannot be limited to detection accuracy, type classification, compilation, or functional preservation alone. Our experiments show that models sometimes misclassify clone types but still generate correct refactorings, while in other cases they classify correctly but fail to remove all duplication. This reinforces the need to treat classification and refactoring as related yet distinct tasks. Researchers may therefore adopt broader evaluation criteria that include clone elimination completeness, incomplete refactoring, expanded refactoring and comprehensive refactoring rates, and cross-model consistency, in addition to traditional correctness checks. Such measures better reflect the practical value of clone management and connect directly to developer concerns about maintainability and trust.

6.3. Implications for Tool Builders

Toward hybrid and retrieval-augmented solutions. Results from RQ3 show that different LLM models exhibit inconsistent behaviors: some achieve higher clone elimination rates but introduce more failures, while others preserve functionality but leave residual duplication. Compared to traditional tools such as JDeodorant or AntiCopyPaster, which demonstrated stability in controlled settings, LLMs offer flexibility but lack consistency. This points toward hybrid solutions in which static or AST-based analyses validate LLM outputs to ensure both completeness and correctness before changes are accepted. Tool builders can focus on supporting tools with dashboards that report metrics such as duplicate reduction, test pass/fail outcomes, and extraction coverage, so developers can make informed decisions about acceptance. A promising avenue is retrieval-augmented generation (RAG), which can ground LLM outputs in project-specific histories of accepted refactorings, coding conventions, and past clone removals. Evidence from the Improta et al. [87] highlights the importance of curating high-quality retrieval corpora, since noisy examples directly increase the likelihood of poor generations. MANTRA [39] further demonstrates that combining RAG with multi-agent review and automated repair yields a higher proportion of compilable, test-passing refactorings compared to single-LLM prompting or IDE baselines. Tool builders can draw on these lessons by coupling retrieval with safeguards and repair pipelines to deliver consistent and trustworthy clone refactoring support.

6.4. Implications for Educators

Teaching the use of AI for clone refactoring. Our study also carries implications for software engineering educators. The results from RQ2 show that LLMs succeed more in same-file refactorings but struggle across files, providing a natural context for teaching students about the strengths and limitations of AI-assisted development. Classroom activities can ask students to apply LLMs to both same-file and cross-file clones, then critically evaluate the outputs using the same criteria applied in our study: completeness of clone elimination, compilation and testing outcomes, and readability of the extracted methods. Such exercises can train students to recognize when LLMs provide reliable support and when caution or alternative approaches are required. Framing LLMs as assistive partners rather than infallible tools equips future developers with a balanced understanding of how to integrate AI responsibly into software maintenance tasks.

7. Threats To Validity

Internal Validity. Several factors could influence our findings. First, we used JDeodorant as the ground truth for evaluating clone classification. However, although JDeodorant is widely used, it relies on external clone detection tools such as Deckard for detecting clones, which introduces certain limitations. Specifically, the quality and usefulness of refactoring largely depend on the clones provided by Deckard, which do not always represent the most optimal set of code duplications. During our manual inspection, we encountered issues such as duplicated clone groups or closely similar groups differing by just one or two lines. To mitigate this limitation, we skipped duplicated clone groups when we manually reviewed the refactoring. Also, the effectiveness of LLMs heavily depends on prompt engineering. Different prompts could lead to different results. To mitigate this, we experimented with zero-shot, one-shot and few-shot learning. Since LLMs can produce slightly different responses each time, especially across different runs, we set the temperature to 0 in our experiments to ensure the outputs were as consistent as possible.

External Validity. Because JFreeChart provided the largest amount of data among available projects when analyzed using JDeodorant, our study was based on JFreeChart, which is a Java-based open-source project. This may cause our results not generalize to other programming languages or other open-source projects. And JFreeChart project may have characteristics such as consistent coding style, mature architecture, or limited use of advanced language features that make it more amenable to automated refactoring. As such, the quality of results observed with GPT-4o may not directly generalize to other codebases. In projects with more complex or less regular code, the effectiveness of refactoring tools could be significantly lower. To complement this, we conducted an additional case study on the JHotDraw project, where we compared multiple LLMs (Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1) alongside JDeodorant. However, the sample size of this

case study was relatively small, with only 20 evaluated instances. Future studies are needed to evaluate the performance across more diverse projects. Another limitation of this study is the reliance on GPT-4o, a proprietary and black-box model whose future availability and behavior are not guaranteed. As OpenAI may modify or deprecate the model or its API, full replication of our results may not be possible. This threat should be considered when interpreting the generalizability of our findings. Also we evaluate GPT-4o, which cannot represent the capabilities of other LLMs. While previous studies suggested that GPT models perform well in clone detection [6, 16, 10], its performance in refactoring may vary from other LLMs.

8. Conclusion

In this study, we systematically evaluated GPT-4o's ability to classify and refactor code clones. We assessed clone classification accuracy, refactoring correctness, compilation success rates, and the use of lambda expressions. Our findings show that GPT-4o performs well in classifying Type-1 clones but struggles with distinguishing Type-2 and Type-3 clones, often misclassifying them. When refactoring clones across different files, GPT-4o encountered significant challenges. These results highlight both the potential and limitations of LLMs in code clone classification and refactoring. While GPT-4o can effectively assist in clone classification and refactoring, it still requires improvements in clone classification accuracy and handling cross-file dependencies. Our study provides valuable insights for researchers seeking to integrate LLMs into software maintenance workflows. To further explore how consistent LLMs are in clone identification and refactoring compared to traditional approaches, we conducted a complementary case study on the JHotDraw project. This study compared several LLMs, such as Gemini 2.5 Pro, GPT-3.5, GPT-4o, GPT-5, DeepSeek-R1, Claude Sonnet 3.7 and Claude Opus 4.1, against JDeodorant and Anti-CopyPaster. Although limited in scale with 20 evaluated instances, the case study revealed clear differences in performance across models, with traditional tools providing stable results while LLMs showed varying levels of effectiveness. These findings emphasize that the capabilities of LLMs are not yet consistent across models, which highlights the importance of broader and more systematic evaluations in future work.

Declaration of generative AI and AI-assisted technologies in the writing process. During the preparation of this work, the authors used the ChatGPT Web interface to improve the language and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z. Li, Y. Ma, Exploring and evaluating hallucinations in llm-powered code generation, arXiv preprint arXiv:2404.00971.

[2] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, S. K. Lahiri, Llm-based test-driven interactive code generation: User study and empirical evaluation, IEEE Transactions on Software Engineering.

[3] M. L. Siddiq, J. C. Da Silva Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat, V. Carvalho Lopes, Using large language models to generate junit tests: An empirical study, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 313–322. doi:10.1145/3661167.3661216.
URL https://doi.org/10.1145/3661167.3661216

[4] S. Kang, J. Yoon, S. Yoo, Large language models are few-shot testers: Exploring llm-based general bug reproduction, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 2312–2323.

[5] J. Cordeiro, S. Noei, Y. Zou, An empirical study on the code refactoring capability of large language models, arXiv preprint arXiv:2411.02320.

[6] M. Khajezade, J. J. Wu, F. H. Fard, G. Rodriguez-Perez, M. S. Shehata, Investigating the efficacy of large language models for code clone detection, in: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, ICPC '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 161–165. doi:10.1145/3643916.3645030.
URL https://doi.org/10.1145/3643916.3645030

[7] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, H. Liu, An empirical study on the potential of llms in automated software refactoring, arXiv preprint arXiv:2411.04444.

[8] A. A. Almatrafi, F. A. Eassa, S. A. Sharaf, Code clone detection techniques based on large language models, IEEE Access.

[9] S. Dou, J. Shan, H. Jia, W. Deng, Z. Xi, W. He, Y. Wu, T. Gui, Y. Liu, X. Huang, Towards understanding the capability of large language models on code clone detection: A survey, arXiv preprint arXiv:2308.01191.

[10] Z. Zhang, T. Saber, Assessing the code clone detection capability of large language models, in: 2024 4th International Conference on Code Quality (ICCQ), 2024, pp. 75–83. doi:10.1109/ICCQ60895.2024.10576803.

[11] E. A. AlOmar, A. Venkatakrishnan, M. W. Mkaouer, C. Newman, A. Ouni, How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations, in: Proceedings of the 21st International Conference on Mining Software Repositories, 2024, pp. 202–206.

[12] F. Batole, A. Bellur, M. Dilhara, M. R. Ullah, Y. Zharov, T. Bryksin, K. Ishikawa, H. Chen, M. Morimoto, S. Motoura, et al., Leveraging llms, ides, and semantic embeddings for automated move method refactoring, arXiv preprint arXiv:2503.20934.

[13] B. Hummel, E. Juergens, L. Heinemann, M. Conradt, Index-based code clone detection: incremental, distributed, scalable, in: 2010 IEEE International Conference on Software Maintenance, IEEE, 2010, pp. 1–9.

[14] M. Fowler, Refactoring: improving the design of existing code, Addison-Wesley Professional, 2018.

[15] X. Qian, E. A. AlOmar, Replication package for "can llms identify and refactor code clones? an empirical study", https://se4airesearch.github.io/clone-refactor-llm/, accessed: 2025-06-30 (2025).

[16] M. B. Moumoula, A. K. Kabore, J. Klein, T. F. Bissyande, Cross-lingual code clone detection: When llms fail short against embedding-based classifier, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 2474–2475. doi:10.1145/3691620.3695335.
URL https://doi.org/10.1145/3691620.3695335

[17] C. K. Roy, J. R. Cordy, R. Koschke, Comparison and evaluation of code clone detection techniques and tools: A qualitative approach, Science of computer programming 74 (7) (2009) 470–495.

[18] S. Ducasse, M. Rieger, S. Demeyer, A language independent approach for detecting duplicated code, in: Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'Software Maintenance for Business Change'(Cat. No. 99CB36360), IEEE, 1999, pp. 109–118.

[19] H. A. Basit, S. Jarzabek, Efficient token based clone detection with flexible tokenization, in: Proceedings of the the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering, 2007, pp. 513–516.

[20] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, L. Bier, Clone detection using abstract syntax trees, in: Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272), IEEE, 1998, pp. 368–377.

[21] N. Tsantalis, D. Mazinanian, G. P. Krishnan, Assessing the refactorability of software clones, IEEE Transactions on Software Engineering 41 (11) (2015) 1055–1090. doi:10.1109/TSE.2015.2448531.

[22] N. Tsantalis, A. Chatzigeorgiou, Identification of extract method refactoring opportunities for the decomposition of methods, Journal of Systems and Software 84 (10) (2011) 1757–1782.

[23] N. Tsantalis, T. Chaikalis, A. Chatzigeorgiou, Ten years of jdeodorant: Lessons learned from the hunt for smells, in: 2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER), IEEE, 2018, pp. 4–14.

[24] S. R. Foster, W. G. Griswold, S. Lerner, Witchdoctor: Ide support for real-time autocompletion of refactorings, in: 2012 34th international conference on software engineering (ICSE), IEEE, 2012, pp. 222–232.

[25] E. Murphy-Hill, C. Parnin, A. P. Black, How we refactor, and how we know it, IEEE Transactions on Software Engineering 38 (1) (2011) 5–18.

[26] S. Negara, N. Chen, M. Vakilian, R. E. Johnson, D. Dig, A comparative study of manual and automated refactorings, in: ECOOP 2013–Object-Oriented Programming: 27th European Conference, Montpellier, France, July 1-5, 2013. Proceedings 27, Springer, 2013, pp. 552–576.

[27] N. Tsantalis, A. Ketkar, D. Dig, Refactoringminer 2.0, IEEE Transactions on Software Engineering 48 (3) (2020) 930–950.

[28] S. Charalampidou, A. Ampatzoglou, A. Chatzigeorgiou, A. Gkortzis, P. Avgeriou, Identifying extract method refactoring opportunities based on functional relevance, IEEE Transactions on Software Engineering 43 (10) (2016) 954–974.

[29] K. Maruyama, Automated method-extraction refactoring by using block-based slicing, in: Proceedings of the 2001 symposium on Software reusability: putting software reuse in context, 2001, pp. 31–40.

[30] O. Tiwari, R. Joshi, Identifying extract method refactorings, in: Proceedings of the 15th Innovations in Software Engineering Conference, 2022, pp. 1–11.

[31] L. Yang, H. Liu, Z. Niu, Identifying fragments to be extracted from long methods, in: 2009 16th Asia-Pacific Software Engineering Conference, IEEE, 2009, pp. 43–49.

[32] D. Pomian, A. Bellur, M. Dilhara, Z. Kurbatova, E. Bogomolov, A. Sokolov, T. Bryksin, D. Dig, Em-assist: Safe automated extractmethod refactoring with llms, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, 2024, pp. 582–586.

[33] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al., Gpt-4o system card, arXiv preprint arXiv:2410.21276.

[34] N. Wadhwa, J. Pradhan, A. Sonwane, S. P. Sahu, N. Natarajan, A. Kanade, S. Parthasarathy, S. Rajamani, Core: Resolving code quality issues using llms, Proceedings of the ACM on Software Engineering 1 (FSE) (2024) 789–811.

[35] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, L. Zhang, ismell: Assembling llms with expert toolsets for code smell detection and refactoring, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1345–1357.

[36] T. Ye, W. Huang, X. Zhang, T. Ma, P. Liu, J. Yin, W. Wang, Llm4effi: Leveraging large language models to enhance code efficiency and correctness, arXiv preprint arXiv:2502.18489.

[37] M. Nashaat, R. Amin, A. H. Eid, R. F. Abdel-Kader, An enhanced transformer-based framework for interpretable code clone detection, Journal of Systems and Software 222 (2025) 112347. doi:https://doi.org/10.1016/j.jss.2025.112347.
URL https://www.sciencedirect.com/science/article/pii/S0164121225000159

[38] R. Inoue, Y. Higo, Improving accuracy of llm-based code clone detection u sing functionally equivalent methods, in: 2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications (SERA), 2024, pp. 24–27. doi:10.1109/SERA61261.2024.10685589.

[39] Y. Xu, F. Lin, J. Yang, N. Tsantalis, et al., Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration, arXiv preprint arXiv:2503.14340.

[40] B. Van Bladel, S. Demeyer, A comparative study of test code clones and production code clones, Journal of Systems and Software 176 (2021) 110940.

[41] E. Kodhai, S. Kanmani, Method-level code clone detection through lwh (light weight hybrid) approach, Journal of Software Engineering Research and Development 2 (2014) 1–29.

[42] M. Ó Cinnéide, L. Tratt, M. Harman, S. Counsell, I. Hemati Moghadam, Experimental assessment of software metrics using automated refactoring, in: Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement, 2012, pp. 49–58.

[43] X. Chen, A. Y. Wang, E. Tempero, A replication and reproduction of code clone detection studies, in: Proceedings of the Thirty-Seventh Australasian Computer Science Conference-Volume 147, 2014, pp. 105–114.

[44] R. Tairas, J. Gray, I. Baxter, Visualization of clone detection results, in: Proceedings of the 2006 OOPSLA workshop on eclipse technology eXchange, 2006, pp. 50–54.

[45] Y. Lin, Z. Xing, Y. Xue, Y. Liu, X. Peng, J. Sun, W. Zhao, Detecting differences across multiple instances of code clones, in: Proceedings of the 36th International Conference on Software Engineering, 2014, pp. 164–174.

[46] OpenAI, Introducing GPT-4o and GPT-4o Mini, accessed: 2025-06-18 (2024).
URL https://openai.com/index/introducing-o3-and-o4-mini/

[47] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al., Starcoder: may the source be with you!, arXiv preprint arXiv:2305.06161.

[48] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, et al., Gemini: a family of highly capable multimodal models, arXiv preprint arXiv:2312.11805.

[49] E. A. AlOmar, A. Ivanov, Z. Kurbatova, Y. Golubev, M. W. Mkaouer, A. Ouni, T. Bryksin, L. Nguyen, A. Kini, A. Thakur, Anticopypaster: extracting code duplicates as soon as they are introduced in the ide, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–4.

[50] E. A. AlOmar, A. Ivanov, Z. Kurbatova, Y. Golubev, M. W. Mkaouer, A. Ouni, T. Bryksin, L. Nguyen, A. Kini, A. Thakur, Just-in-time code duplicates extraction, Information and Software Technology 158 (2023) 107169.

[51] E. A. AlOmar, B. Knobloch, T. Kain, C. Kalish, M. W. Mkaouer, A. Ouni, Anticopypaster 2.0: Whitebox just-in-time code duplicates extraction, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, 2024, pp. 84–88.

[52] E. A. AlOmar, J. Ashkenas, R. Feliciano, M. Angelakos, D. Haralamppopoulos, X. Qian, M. W. Mkaouer, A. Ouni, Anticopypaster 3.0: Just-in-time clone refactoring, ACM Transactions on Software Engineering and Methodology.

[53] M. Palatucci, D. Pomerleau, G. E. Hinton, T. M. Mitchell, Zero-shot learning with semantic output codes, Advances in neural information processing systems 22.

[54] M. Wu, M. Mosse, N. Goodman, C. Piech, Zero shot learning for code education: Rubric sampling with deep learning inference, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 782–790.

[55] L. Fei-Fei, R. Fergus, P. Perona, One-shot learning of object categories, IEEE transactions on pattern analysis and machine intelligence 28 (4) (2006) 594–611.

[56] L. Fe-Fei, et al., A bayesian approach to unsupervised one-shot learning of object categories, in: proceedings ninth IEEE international conference on computer vision, IEEE, 2003, pp. 1134–1141.

[57] T. Kamiya, S. Kusumoto, K. Inoue, Ccfinder: A multilinguistic token-based code clone detection system for large scale source code, IEEE transactions on software engineering 28 (7) (2002) 654–670.

[58] L. Jiang, G. Misherghi, Z. Su, S. Glondu, Deckard: Scalable and accurate tree-based detection of code clones, in: 29th International Conference on Software Engineering (ICSE'07), IEEE, 2007, pp. 96–105.

[59] C. K. Roy, J. R. Cordy, Nicad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization, in: 2008 16th iEEE international conference on program comprehension, IEEE, 2008, pp. 172–181.

[60] N. Tsantalis, Jdeodorant, https://github.com/tsantalis/jdeodorant, [Online; accessed 2025] (2011).

[61] E. A. AlOmar, M. W. Mkaouer, A. Ouni, Behind the intent of extract method refactoring: A systematic literature review, IEEE Transactions on Software Engineering 50 (4) (2024) 668–694.

[62] W. G. Griswold, D. Notkin, Automated assistance for program restructuring, ACM Transactions on Software Engineering and Methodology (TOSEM) 2 (3) (1993) 228–269.

[63] A. V. Zarras, T. Vartziotis, P. Vassiliadis, Navigating through the archipelago of refactorings, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 2015, pp. 922–925.

[64] D. Silva, N. Tsantalis, M. T. Valente, Why we refactor? confessions of github contributors, in: Proceedings of the 2016 24th acm sigsoft international symposium on foundations of software engineering, 2016, pp. 858–870.

[65] N. Tsantalis, V. Guana, E. Stroulia, A. Hindle, A multidimensional empirical study on refactoring activity., in: CASCON, 2013, pp. 132–146.

[66] E. Murphy-Hill, A. P. Black, Breaking the barriers to successful refactoring: observations and tools for extract method, in: Proceedings of the 30th international conference on Software engineering, 2008, pp. 421–430.

[67] S. Charalampidou, E.-M. Arvanitou, A. Ampatzoglou, P. Avgeriou, A. Chatzigeorgiou, I. Stamelos, Structural quality metrics as indicators of the long method bad smell: An empirical study, in: 2018 44th Euromicro Conference on software engineering and advanced applications (SEAA), IEEE, 2018, pp. 234–238.

[68] S. Thy, A. Costea, K. Gopinathan, I. Sergey, Adventure of a lifetime: Extract method refactoring for rust, Proceedings of the ACM on Programming Languages 7 (OOPSLA2) (2023) 658–685.

[69] E. AlOmar, M. W. Mkaouer, A. Ouni, Can refactoring be self-affirmed? an exploratory study on how developers document their refactoring activities in commit messages, in: 2019 IEEE/ACM 3rd International Workshop on Refactoring (IWoR), IEEE, 2019, pp. 51–58.

[70] D. van der Leij, J. Binda, R. van Dalen, P. Vallen, Y. Luo, M. Aniche, Data-driven extract method recommendations: a study at ing, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 1337–1347.

[71] N. Yoshida, S. Numata, E. Choiz, K. Inoue, Proactive clone recommendation system for extract method refactoring, in: 2019 IEEE/ACM 3rd International Workshop on Refactoring (IWoR), IEEE, 2019, pp. 67–70.

[72] J. P. S. Alcocer, A. S. Antezana, G. Santos, A. Bergel, Improving the success rate of applying the extract method refactoring, Science of Computer Programming 195 (2020) 102475.

[73] K. Hotta, Y. Sano, Y. Higo, S. Kusumoto, Is duplicate code more frequently modified than non-duplicate code in software evolution? an empirical study on open source software, in: Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE), 2010, pp. 73–82.

[74] Y. Higo, S. Kusumoto, K. Inoue, A metric-based approach to identifying refactoring opportunities for merging code clones in a java software system, Journal of Software Maintenance and Evolution: Research and Practice 20 (6) (2008) 435–461.

[75] E. A. AlOmar, T. Wang, V. Raut, M. W. Mkaouer, C. Newman, A. Ouni, Refactoring for reuse: an empirical study, Innovations in Systems and Software Engineering 18 (1) (2022) 105–135.

[76] E. A. AlOmar, P. T. Rodriguez, J. Bowman, T. Wang, B. Adepoju, K. Lopez, C. Newman, A. Ouni, M. W. Mkaouer, How do developers refactor code to improve code reusability?, in: International Conference on Software and Software Reuse, Springer, 2020, pp. 261–276.

[77] R. Morales, Z. Soh, F. Khomh, G. Antoniol, F. Chicano, On the use of developers' context for automatic refactoring of software anti-patterns, Journal of systems and software 128 (2017) 236–251.

[78] F. Khomh, M. D. Penta, Y.-G. Guéhéneuc, G. Antoniol, An exploratory study of the impact of antipatterns on class change-and fault-proneness, Empirical Software Engineering 17 (3) (2012) 243–275.

[79] F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, Do they really smell bad? a study on developers' perception of bad code smells, in: 2014 IEEE International Conference on Software Maintenance and Evolution, IEEE, 2014, pp. 101–110.

[80] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, A. De Lucia, On the diffuseness and the impact on maintainability of code smells: a large scale empirical investigation, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 482–482.

[81] W. Oizumi, A. C. Bibiano, D. Cedrim, A. Oliveira, L. Sousa, A. Garcia, D. Oliveira, Recommending composite refactorings for smell removal: Heuristics and evaluation, in: Proceedings of the XXXIV Brazilian Symposium on Software Engineering, 2020, pp. 72–81.

[82] M. Ó. Cinnéide, D. Boyle, I. H. Moghadam, Automated refactoring for testability, in: 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops, IEEE, 2011, pp. 437–443.

[83] M. Harman, Refactoring as testability transformation, in: 2011 IEEE Fourth International conference on software testing, verification and validation workshops, IEEE, 2011, pp. 414–421.

[84] A. Hora, R. Robbes, Characteristics of method extractions in java: A large scale empirical study, Empirical Software Engineering 25 (3) (2020) 1798–1833.

[85] E. A. AlOmar, A. Peruma, M. W. Mkaouer, C. Newman, A. Ouni, M. Kessentini, How we refactor and how we document it? on the use of supervised machine learning algorithms to classify refactoring documentation, Expert Systems with Applications 167 (2021) 114176.

[86] N. Tsantalis, D. Mazinanian, S. Rostami, Clone refactoring with lambda expressions, in: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), IEEE, 2017, pp. 60–70.

[87] C. Improta, R. Tufano, P. Liguori, D. Cotroneo, G. Bavota, Quality in, quality out: Investigating training data's role in ai code generation, arXiv preprint arXiv:2503.11402.